Documentation for **ProteinEvolverABC**

Estimation of Recombination and Substitution rates in alignments of protein sequences by approximate Bayesian computation

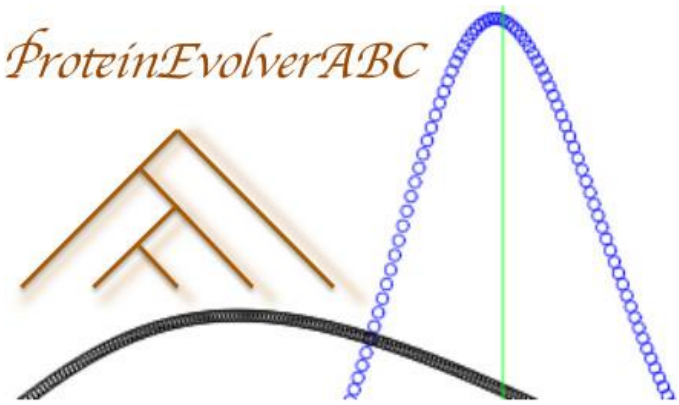
Current version is 2.0

© 2020-2026.

Elena Pazos-Linares, Cristina Landa, Ugo Bastolla, Miguel Arenas

marenas@uvigo.es

April 16, 2026



Contents

Disclaimer 3

Credits 3

1. Purpose 3

1.1. Parameters to estimate 4

2. Software required for ProteinEvolverABC 5

2.1. Software installations 5

2.2. Execute ProteinEvolverABC 5

3. ProteinEvolverABC usage 6

3.1. The settings input file 7

3.1.1. Available prior distributions 8

3.1.2. Simulation phase 8

3.1.3. Estimation phase 13

3.2. The target alignment of protein sequences 14

3.3. The protein structure 15

3.4. The sequence of the protein structure 15

3.5. Screen information 15

3.4. Output files 32

3.4.1. Output files from simulations 32

3.4.2. Summary statistics from the target multiple alignment of protein sequences 33

3.4.3. Output files from the estimations 33

3.5. Re-analysing data 34

3.6. Examples 35

3.7. Message errors and recommendations 36

4. Models and methods 39

4.1. Evolutionary models and computer simulations 39

4.2. Summary statistics 39

4.3. ABC estimation approaches 40

4.3.1. The rejection approach 40

4.3.2. The regression approach 41

4.4. ProteinEvolverABC performance 41

4.5. Assumptions and limitations 42

5. Acknowledgments 42

6. References 42

Disclaimer

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version (at your option) of the License. This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 59 Temple Place – Suite 330, Boston, MA 02111-1307, USA.

Credits

This program was developed at,

- *CINBIO, University of Vigo, 36310 Vigo, Spain.*
- *Department of Biochemistry, Genetics and Immunology, University of Vigo, 36310 Vigo, Spain.*
- *Centre for Molecular Biology Severo Ochoa (CSIC-UAM), 28049, Madrid, Spain.*

- The first version of this framework was supported by the Spanish Ministry of Economy and Competitiveness and Ministry of Science and Innovation through the Grants [RYC- 2015-18241] and [PID2019-107931GA-I00/AEI/10.13039/501100011033].
- The second (current) version of this framework was partially supported by the grants [PID2023-151032 NB-C21] and [PID2023-151032NB-C22] funded by the Spanish Ministry of Science MCIU/the Spanish Agency of Research AEI/10.13039/501100011033/ and FEDER, UE.

1. Purpose

ProteinEvolverABC is an evolutionary framework to estimate recombination and substitution rates by the approximate Bayesian computation (ABC) approach [1, 2] from protein sequence alignments. *ProteinEvolverABC* is designed to be run mainly on Linux OS command line, and it is freely available from https://github.com/MiguelArenas/ProteinEvolverABC.

The user can specify prior distributions for the parameters to estimate (i.e. recombination and substitution rates), but also for other evolutionary parameters that will be treated as nuisance parameters, like amino acid frequencies, proportion of invariable sites (+I) or heterogeneity change across sites according to a gamma distribution (+G).

The computer simulations are performed via the coalescent program *ProteinEvolverProtABC*, an adapted version of the simulator *ProteinEvolver* (https://github.com/MiguelArenas/proteinevolver) [3] to ABC. Importantly, the second version of the framework considers evolutionary constraints on the protein structure through several structurally constrained substitution (SCS) models such as SSCPE (Stability and Structure Constrained Protein Evolution models, which includes a series of SCS models [4]) and other SCS models that were successfully applied to infer ancestral protein sequences [5].

ProteinEvolverABC implements a total of 20 summary statistics, some of them have been shown able to extract information from DNA data to accurately estimate the parameters of interest [6, 7]. Indeed, some summary statistics capture physicochemical properties from the protein structure (i.e., protein folding stability) [8, 9], as they can also inform about substitution and recombination processes at the level of the protein structure [8, 10, 11]. Conveniently, *ProteinEvolverABC* can run the simulations on parallel, according to user specifications, to save computer time.
Different ABC estimation methods are implemented in *ProteinEvolverABC*, in particular a rejection algorithm [12] and a weighted multiple linear regression algorithm as suggested in [1]. For both algorithms the user can specify the number of simulations to use, the number of simulations to retain (tolerance of the rejection algorithm) and which summary statistics to use. For the regression algorithm, the user can also specify whether to transform the parameters and which transformation to use (log-transformation or logit-transformation), and whether to correct for the heteroscedastic variance of the parameters. *ProteinEvolverABC* provides also several diagnostic plots to assess qualitatively how well the model fitted the data, namely: histograms of the summary statistics of the retained simulations; scatterplots of the summary statistics and each parameter; and a scatterplot of the two principal components calculated from the summary statistics of the simulations. In all the plots the values for the target protein alignment data are superimposed.

1.1. Parameters to estimate

The parameters that can be estimated by *ProteinEvolverABC* are recombination and substitution rates. Their technical details are shown below.

The population recombination rate per site (amino acid site) per generation (ρ) is described by the following equation (1):

$$\rho = 2zNrl \qquad (1)$$

where z can be 1 or 2 for haploid or diploid data, N is the effective population size, r is the recombination rate per site, and l is the sequence length in amino acids.
In nature, ρ commonly ranges between 0 (no recombination) to 100 [e.g., 6, 13, 14]. Given ρ and the number of sequences one can obtain the expected number of recombination events by:

$$R[R(n)] = Rho \sum_{j=1}^{n-1} \frac{1}{j} \qquad (2)$$

where n is the number of sequences. For example, for 10 sequences, 0, 2.72, 10.87, 43.49 and 86.97 are the number of recombination events expected for ρ values of 0, 1, 4, 16 and 32, respectively. Importantly, ρ is frequently estimated from DNA sequences [e.g., 6, 7, 15, 16-18] and hence a goal of this framework is the estimation of ρ in protein sequences.

The population amino acid substitution rate per site per generation (θ) is described by the following equation:

$$\theta = 2zN\mu L \qquad (3)$$

where z is 1 or 2 for haploid or diploid data, N is the effective population size, μ is the substitution rate per amino acid and L is the sequence length in amino acids. Common values for θ were found between 50 and 300 [e.g., 6, 19, 20]. A θ = 0 means that there are not substitutions (fixed mutations) in the alignment. As an example, for 25

sequences, evolved without recombination, a θ = 50, 100 and 200 can provide alignments with sequence identity of 0.92, 0.84 and 0.73, respectively.

2. Software required for *ProteinEvolverABC*

2.1. Software installations

The *ProteinEvolverABC* framework runs on the command line and requires that Perl and R are installed. Perl can be downloaded from http://www.perl.org/ and R from http://www.r-project.org/. *ProteinEvolverABC* also requires the following R libraries: *lattice*, *MCMCpack*, *ape*, *graphics* and *abc*. These libraries can be installed by the command: install.packages("*library_name*"), see further details in http://cran.r-project.org/doc/manuals/R-admin.html#Installing-packages. For example,

```
install.packages("lattice")
```

The folder "source" contains the source codes of the simulator *ProteinEvolverProtABC*, the *SSCPE* framework that includes the SSCPE substitution models [4], the *ProtASR2* framework that includes the Meanfield (MF) and Wild-Type (WT) SCS models [5], all of them used to simulate protein sequence evolution. It also includes the software *PhiTest* [21], which includes some rapid recombination tests. All these programs are compiled with the compiler *gcc* (details below). A perl script "*PathsProt_evol.pl*" is also included to manage paths.
The folder "scripts" contains several scripts required to execute *ProteinEvolverABC*.
All this material must NOT be modified by the user at any time as it is required to compile and run the program.

2.2. Execute ProteinEvolverABC

First, *ProteinEvolverABC* must be compiled in the command line through a Makefile that is placed on the main directory,

```
make all
```

Thus, all the embedded tools included in the framework will be compiled and the executable files will be placed into the folder "*bin*".

The execution of *ProteinEvolverABC* consists of the following 5 separated steps.
Importantly, all the input files (Settings.txt, the multiple alignment of protein sequences and, if included, the files with the protein structure and its sequence) must be located in the main directory.

1. Read the input files and **sample from the prior distributions to later perform the simulations**. It also provides information about the prior distributions. Type,

```
perl ProteinEvolverABC_Phase1.pl Settings.txt
```

2. If required, generate the exchangeability matrix (or matrices) for the simulation step and **compute the summary statistics for the input alignment.**

```
perl ProteinEvolverABC_Phase2.pl Settings.txt
```

3. Run the simulations. Optionally, the simulations can run in parallel to save time. Type,

```
perl ProteinEvolverABC_Phase3.pl Settings.txt
```

4. Compute summary statistics for the simulated data. Type,

```
perl ProteinEvolverABC_Phase4.pl Settings.txt
```

5. Compute estimations of substitution and recombination rates (posterior distributions), provides information about the summary statistics and posterior distributions. This execution can be repeated with different ABC settings (see details later). Type,

```
perl ProteinEvolverABC_Phase5.pl Settings.txt
```

At each step, running information is written on the screen (see details later).
The time required for a full execution depends on the size of the input data, the specified models and parameters, and the capacity of the used computer. In particular, steps 3 and 4 may take a considerable amount of time especially when performing a large number of simulations, when the protein dataset includes a large number and long sequences, or when considering high recombination rates. Indeed, including structural information in the simulations and/or in the summary statistics can increase the required computer time. However, the step 3 can be run in parallel if the user specifies this in the settings file and works with a machine presenting more than one processor. If the specified simulation settings are unexpected (e.g., too high recombination rates or) the analysis may fail (see section about "Message Errors"). Indeed, the step 3 generates a number of simulated sequence alignments in the working directory, requiring disk space. In the step 4, these simulations can be removed or saved as a zip file depending on user specifications.

3. *ProteinEvolverABC* usage

The main input file of the program is the settings file, where the user must specify settings such as prior distributions for evolutionary parameters or the estimation method. However, other input files can be required.
A standard run of *ProteinEvolverABC* framework requires the following files from the user:

- **Settings file**. This input file must contain all the specifications for the estimation. It should be made by the user (i.e., modifying a Settings.txt file provided in the examples) with the desired specifications (details below). Note that the name of this file must be considered to run *ProteinEvolverABC* (e.g., *perl ProteinEvolverABC_Phase1.pl Settings.txt*).
- **Multiple alignment of protein sequences**. The protein sequence alignment that will be analyzed must be provided by the user in sequential phylip format (.phy). **This file should be placed in the settings file directory, which is the main working directory, and its name should be specified in the settings file.** See details below.
- **Protein structure**. Optional for simulations or calculation of summary statistics that consider structural constraints. A protein structure (i.e., a file from the Protein Data Bank, PDB; PDB file), which should be representative of the structure of the protein sequences included in the multiple sequence alignment, is required by the SCS models of protein evolution (either SSCPE, ProtASR2_MF or ProtASR2_WT) and by the structural summary statistics. This file should be placed in the directory of the settings file, which is the main working directory, and its name must be specified in the settings file. There are

several ways to obtain a protein structure representative of several protein sequences: (*i*) Certain databases of protein families include representative protein structures, (*ii*) the protein structure can be predicted from one or more protein sequences by the traditional homology modeling [22, 23], (*iii*) the protein structure can be predicted from one or more protein sequences by deep learning approaches [24].

- **Amino acid sequence of the protein structure**. Optional for simulations or calculation of summary statistics that consider structural constraints. The sequence of the protein structure is assigned to the root of the evolutionary history (the Grand Most Recent Common Ancestor, GMRCA [25, 26]; SeqGMRCA file) to perform the simulation of protein evolution (Phase 3) following [3]. This file should be placed in the directory of the settings file, which is the main working directory, and its name must be specified in the settings file. Details are shown below.

Several examples of all these input files for standard *ProteinEvolverABC* analyses are provided in the package (folder "examples").

The user is advised to create a folder in his/her desired location (e.g. his own home folder) and copy there the required files and all *ProteinEvolverABC* material (including executable files and folders).
After concluding a sucessfull run of *ProteinEvolverABC*, two folders are created in the working directory, the "MainOutputs" and the "OtherOuputs" folders (see details later). Since the estimation phase (phase 5) is quite fast in comparison with the simulation phase, the user is advised to explore different settings for the ABC method if desired (see details later and "re-analyzing the data"), without having to run again the previous phases.

3.1. The settings input file

The settings input file must contain all the information required to perform the analysis and should be carefully checked by the user since mistakes may affect results. Indeed, error messages will be displayed on the screen if something is incorrectly specified and may suggest stop the execution by typing CTRL+C.
The file consists of two main blocks, the simulation phase and the estimation phase.

Important notes when dealing with the settings file
- A parameter value must be introduced in the line after the corresponding parameter description, otherwise such a parameter will be considered as "not specified" or "incorrect" (see examples below).
- Do not modify the parameter description (it is used to identify the parameter).
- Some parameters are mandatory and must be specified, these parameters contain an astherisk "*" (see below).
- Some parameters require a prior distribution (see details later), this is a distribution within which the estimation should fall (the range of the prior distribution should include the estimation; for example, if the user believes that the recombination rate can be 0 then the prior distribution for the recombination rate should include 0).
- Text that should not be read by the program (i.e., certain optional parameters) can be "turned-off" by indicating an "#" at the beginning of that line.

3.1.1. Available prior distributions

Several distributions are included in *ProteinEvolverABC* to simulate protein data under different evolutionary scenarios (Table 1). In addition, most of them can be truncated at lowest and highest values. However, note that some parameters must be specified by an integer number (e.g., generation time) while others by a non-integer number (e.g., amino acid frequencies). Therefore, not all distributions can be applied to any parameter. Details for each parameter are described in the following subsections. Indeed, several distributions can be truncated at both sides of the distribution according to user specifications (Table 1).

Table 1. Available distributions in *ProteinEvolverABC*.

Distribution	Description	Truncated	Examples
fix	Fixed value (integer or non-integer)	n.a.	fix 4; fix 0.7
Uniform (uniform)	Random between two values (integer or non-integer): lowest highest	n.a.	uniform 1.0e-8 1.0e-5; uniform 2e-9 5e-6; uniform 0 3
Normal (norm)	Normal distribution (mean, sd)	t # #	norm 1.0e-8 1.0e-5; norm 1.0e-8 1.0e-5 t 1.0e-9 1.0e-6
Exponential (exp)	Exponential distribution (rate)	t # #	exp 1.0e-7; exp 1.0e-7 t 1.0e-8 1.0e-6
gamma	Gamma distribution (shape, rate "1/scale")	t # #	gamma 1.0e-7 5.0e-7; gamma 1.0e-7 5.0e-7 t 2.0e-7 1.0e-6
beta	Beta distribution (shape1, shape 2)	t # #	beta 1.0e-7 5.0e-7; beta 1.0e-7 5.0e-7 t 2.5e-7 1.0e-6
dirichlet	Dirichlet distribution (alpha "vector")	n.a.	dirichlet 1 1 1 1; dirichlet 1 1 1 1 1 1

3.1.2. Simulation phase

General settings for the simulation: the Settings.txt file

- **Name of the file with the target protein sequences alignment.** This specification is mandatory. Only specify the filename (i.e. no pathway) since the target alignment file needs to be placed in the same pathway as the settings file. See section 4.2 concerning the format of the multiple sequences alignment. For example,

```
 ### Target alignment file ###          # phylip format
*NameOfPhylipFile= ProtSeq1.phy
```

- **Number of simulations.** This parameter is mandatory. We recommend at least 50,000 simulations to obtain accurate estimates. With more simulations, the results could be better. However, the number of simulations required to obtain accurate estimates depends on many factors, especially the target multiple sequence alignment (data with large molecular diversity could require more simulations because the parametric landscape could be more irregular and should be more sampled) and the range covered by the prior distributions (wide ranges are expected to require more simulations to cover the studied parameters landscape). For example,

```
### Total number of simulations ###
*NumberOfSimulations=500
```

- **Consideration of indels.** This parameter is mandatory. The user specifies whether indels (insertion-deletion regions, gaps) should be ignored (by default and recommended) or considered as a new state. This decision can affect the summary statistics if there are indels in the multiple sequence alignment. For example,

```
# Consideration of indels. 0 (indels are ignored), 1 (indels are considered as
a new state)
*Indels=0
```

- **Number of processors to run the simulations in parallel on a machine.** This parameter is optional (by default simulations run on only one processor). This parallelization works on machines using Linux OS with shared memory. Ideally, one should specify the number of available processors in the computer. On a Linux OS, the number of available processors can be provided by,

```
grep -c ^processor /proc/cpuinfo
```

The number of processors must be specified in the settings file. For example,

```
# Number of available processors to run the simulations in parallel. 1 by
default
NumberOfProcessors=8
```

- **Save simulated data.** This parameter is mandatory. "0" the simulated data is not saved (recommended, simulated data may require a lot of space in the hard disk of the computer), "1" the simulated data is saved. Note that even if choosing not to save the simulated data, simulated data is created and saved temporally since this data is required for calculating the summary statistics in the following step. For example,

```
# Save simulated data. 0 (No), 1 (Yes, but it requires space in the disk)
* SaveSimulations=0
```

- **Show running information.** This parameter is mandatory. "0" reduces the amount of information printed on the screen during the simulation and computation of summary statistics phases. "1" shows all the information printed on the screen during the simulation and computation of summary statistics phases. Applying "0" or "1" does not change the final results, it only affects the amount of information shown on the screen during the execution. It is recommended specify "0" to save computer time (printing information on the screen requires more computer time). For example,

```
# Show running information (simulations and summary statistics) on the screen.
0 (No), 1 (Yes, but it slows down the running time)
*ShowInformationScreen=0
```

Demographic settings

- **Haploid/diploid simulated data.** Haploid is defined with a value of 1 and diploid with a value of 2. This parameter is mandatory. For example,

```
# Haploid or Diploid data (haploid=1, diploid=2)
*Haploid/Diploid=2
```

- **Effective population size (*N*).** The parameter value must be an integer and it is mandatory. For example,

```
# Population size (i.e., 1000).
*PopulationSize=1000
```

Longitudinal sampling

- **Sampling at different times.** This parameter is optional. The user can specify the time at which the tip nodes of the tree (i.e. samples) were sampled in years. In the

example, 4 sampling times are specified: sampled in 1995 - sequences 1 to 10; sampled in 2003 - sequences 11 to 16; sampled in 1997 - sequences 17 to 26; sampled in 2001: sequences 7 and 8. Note that this option does not work if a deme converges, backwards in time, before the last sampling time. For example,

```
# Logitudinal sampling. Requires GenerationTime.
DatedTips=4 1995 1 10 2003 11 16 1997 17 26 2001 27 29
```

- **Generation time.** This parameter is optional. The user can specify the time for each generation. The parameter value can be fixed (fix) or sampled from a uniform distribution (uniform). For example,

```
# Generation time. fix, uniform; i.e., uniform 500 1000
GenerationTime=fix 1200
```

Recombination

- **Recombination rate per site.** This parameter is mandatory and will be estimated. Distributions allowed: fix, uniform, norm(t), exp(t), gamma(t), beta(t).

Important: Note that ρ (the population recombination rate, which can be calculated according to the formula 1) should not be higher than 150, or there might be out-of-memory errors during simulation resulting from considering "infinite" nodes in the ancestral recombination graph (ARG [25, 26]). For example,

```
#Recombination rate per site. fix, uniform, gamma, beta, normal, exponential;
i.e., gamma 0.02 0.5 t 1.5e-07 9.3e-07. -PARAMETER TO BE ESTIMATED-
*RecombinationRate=uniform 0 5.56e-6
```

Protein evolution and substitution model

Amino acid substitution rate per site. This parameter is mandatory and will be estimated. Distributions allowed: fix, uniform, norm(t), exp(t), gamma(t), beta(t). It is highly recommended, when previous knowledge is available, to narrow the interval in order to concentrate the sampling in the region the real value might be. It is important to consider that a substitution rate of 0 (or θ below 50, the population substitution rate, which can be calculated according to the formula 3) will lead to equal or almost equal sequences as no or few substitution events are incorporated, disallowing accurate estimates of the recombination rate. For example,

```
# Amino acid substitution rate. i.e., fix 7.0e-6. -PARAMETER TO BE ESTIMATED-
*SubstitutionRate=uniform 0 1.67e-4
```

- **Model of amino acid substitution.** The user must specify one of the following substitution models of protein evolution:
  - **An empirical substitution model** (global exchangeability matrix) [27]: Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb, HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. Example,

```
# Model of amino acid substitution (several options): (1) Empirical model
(global exchangeability matrix): Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb,
HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. (2) Given
global model: UserEAAM. (3) Given site-specific model: UserEAAMsites. (4)
SSCPE model: SSCPE (requires information in the following lines). (5) ProtASR2
SCS models: ProtASR2_WT, ProtASR2_MF (requires information in the following
lines)
*SubstitutionModel=JTT
#pdb=8C0G.pdb
#chain=A
#pdbsequence=seqGMRCA
```

- A given **global exchangeability matrix**: UserEAAM. This is an exchangeability matrix that is applied to all the protein sites. It must be provided as an additional input file with name UserEAAM. This file should include the exchangeability matrix in *PAML* format [27, 28], and it is explained with details in the documentation of the simulator *ProteinEvolver* [1] and it is included in examples. Example,

```
# Model of amino acid substitution (several options): (1) Empirical model
(global exchangeability matrix): Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb,
HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. (2) Given
global model: UserEAAM. (3) Given site-specific model: UserEAAMsites. (4)
SSCPE model: SSCPE (requires information in the following lines). (5) ProtASR2
SCS models: ProtASR2_WT, ProtASR2_MF (requires information in the following
lines)
*SubstitutionModel=UserEAAM
#pdb=8C0G.pdb
#chain=A
#pdbsequence=seqGMRCA
```

- A given file with **site-specific exchangeability matrices**: UserEAAMsites. This includes an exchangeability matrix for each protein site. This is a site-specific, user-defined model that must be provided as an additional input file with name UserEAAMsites. The exchangeability matrices should be indicated in *PAML* format [27, 28], and are explained with details in the documentation of the simulator *ProteinEvolver* [1], and it is included in examples. Example,

```
# Model of amino acid substitution (several options): (1) Empirical model
(global exchangeability matrix): Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb,
HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. (2) Given
global model: UserEAAM. (3) Given site-specific model: UserEAAMsites. (4)
SSCPE model: SSCPE (requires information in the following lines). (5) ProtASR2
SCS models: ProtASR2_WT, ProtASR2_MF (requires information in the following
lines)
*SubstitutionModel=UserEAAMsites
#pdb=8C0G.pdb
#chain=A
#pdbsequence=seqGMRCA
```

- A SCS SSCPE model implemented in the framework SSCPE [4]. These models consider evolutionary constraints from the protein structure. They require complementary information in the following lines.

For example,

```
# Model of amino acid substitution (several options): (1) Empirical model
(global exchangeability matrix): Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb,
HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. (2) Given
global model: UserEAAM. (3) Given site-specific model: UserEAAMsites. (4)
SSCPE model: SSCPE (requires information in the following lines). (5) ProtASR2
SCS models: ProtASR2_WT, ProtASR2_MF (requires information in the following
lines)
*SubstitutionModel=SSCPE
pdb=8C0G.pdb
chain=A
pdbsequence=seqGMRCA
```

- A SCS model implemented in the framework *ProtASR2* [5]: It includes the models wild-type "ProtASR2_WT" and meanfield "ProtASR2_MF". These models consider evolutionary constraints from the protein structure. They require complementary information in the following lines.

For example,

```
# Model of amino acid substitution (several options): (1) Empirical model
(global exchangeability matrix): Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb,
HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. (2) Given
```

```
global model: UserEAAM. (3) Given site-specific model: UserEAAMsites. (4)
SSCPE model: SSCPE (requires information in the following lines). (5) ProtASR2
SCS models: ProtASR2_WT, ProtASR2_MF (requires information in the following
lines)
*SubstitutionModel=ProtASR2_WT
pdb=8C0G.pdb
chain=A
pdbsequence=seqGMRCA
```

- **Additional information about the protein structure** for the SCS models (SSCPE, ProtASR2_WT, ProtASR2_MF). This is mandatory for the indicated SCS models, for the other models it should not be indicated.
  - Name of the PDB file (*pdb*). Only specify the filename (i.e. no pathway) since this file should be placed in the same pathway as the settings file.
  - Chain of the protein structure (*chain*). The used chain of the PDB file.
  - Sequence of the protein structure (*pdbsequence*). Only specify the filename (i.e. no pathway) since this file should be placed in the same pathway as the settings file. The format of this file is indicated later.

For example,

```
# Model of amino acid substitution (several options): (1) Empirical model
(global exchangeability matrix): Blosum62, CpRev, Dayhoff, DayhoffDCMUT, HIVb,
HIVw, JTT, JonesDCMUT, LG, Mtart, Mtmam, Mtrev24, RtRev, VT, WAG. (2) Given
global model: UserEAAM. (3) Given site-specific model: UserEAAMsites. (4)
SSCPE model: SSCPE (requires information in the following lines). (5) ProtASR2
SCS models: ProtASR2_WT, ProtASR2_MF (requires information in the following
lines)
*SubstitutionModel=SSCPE
pdb=8C0G.pdb
chain=A
pdbsequence=seqGMRCA
```

- **Amino acid frequencies.** Frequencies for each amino acid site along sequences. Distributions allowed: fix or dirichlet. This parameter is optional. If it not specified, for an empirical substitution model, the program assumes equally distributed frequencies (all fix with value 0.05). Other substitution models (i.e., user-specified exchangeability matrices or SCS models) may already have included the amino acid frequencies. For example,

```
# Amino acid frequencies. fix or dirichlet. By default equally distributed
frequencies. i.e., dirichlet 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
*AminoacidFrequencies=fix 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
```

- **Rate of heterogeneity across sites (+G).** Distributions allowed: fix, uniform, norm(t), exp(t), gamma(t), beta(t). This parameter is optional. For example,

```
# Rate of heteregeneity across sites, +G. fix, uniform, gamma, beta, normal,
exponential; i.e., fix 0.6
RateHetSites=uniform 0.76 0.90
```

- **Proportion of invariable sites (+I).** Distributions allowed: fix, uniform, norm(t), exp(t), gamma(t), beta(t). This parameter is optional. For example,

```
# Proportion of invariable sites, +I. fix, uniform, gamma, beta, normal,
exponential; i.e., exponential 0.002 t 0 1.0
#PropInvSites=uniform 0.3 0.5
```

3.1.3. Estimation phase

All these parameters are mandatory. The estimation phase (through *ProteinEvolverABC_Phase5.pl*) can be repeated any number of times allowing to perform estimations under different simulated data and analytical methods (see details later) without having to repeat the phases 1-4.

Summary statistics

- **Information for summary statistics that consider the protein structure.** This is optional. The summary statistics 17-20 (which consider the protein structure, see the next parameter below) requires the specification of the following information:
  - Name of the PDB file (*SSpdb*). Only specify the filename (i.e. no pathway) since this file should be placed in the same pathway as the settings file.
  - Chain of the protein structure (*SSchain*). The used chain of the PDB file.
  - Sequence of the protein structure (*SSpdbsequence*). Only specify the filename (i.e. no pathway) since this file should be placed in the same pathway as the settings file. The format of this file is indicated later.

It is important to mention that sequences simulated under a model that ignores constraints on the protein structure (i.e., an empirical substitution model or user-specified exchangeability matrix/matrices) can include these summary statistics, of course if the corresponding files (protein structure and sequence of the protein structure) are available.

```
# Protein structure for structurally constrained summary statistics, if specified it applies (optional)
SSpdb=8C0G.pdb
SSchain=A
SSpdbsequence=seqGMRCA
```

- **Summary statistics to use.** The user must choose which summary statistics to use for the ABC estimation by specifying their numeric identifiers (see section later "Models and Methods" for further details). By default, it is recommended to specify the 16 summary statistics implemented in the program for substitution models that ignore the protein structure, and the 20 summary statistics for substitution models that consider the protein structure. However, changing the selected summary statistics could affect the results, and thus exploring the influence of summary statistics on the accuracy of the estimation (i.e., see the associated article) can be convenient. For example,

```
# Summary statistics to use. See documentation for details (17-20: structurally constrained summary statistics)
*SummaryStatistics= 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

ABC estimation

- **ABC iterations.** Number of simulations to consider in the ABC analysis. Unless testing the methodology, the user is advised to use all the simulations performed. Note that the number of simulations to use must be obviously equal or lower than the number of simulations performed (specified in *NumberOfSimulations*). For example,

```
#ABC iterations. Number of simulations to consider (Iterations <= NumberOfSimulations)
*ABCIterations=250
```

- **ABC tolerance.** Number of selected simulations (those closer to the study data) to retain in the ABC procedure. Note that the tolerance considered must be obviously lower than the total number of simulations (specified in *NumberOfSimulations*). A

tolerance of 100 can be enough but the best tolerance varies among multiple sequence alignments. It is recommended to explore different values of this parameter. For example,

```
#ABC tolerance. Number of simulations closest to real data to retain in the
ABC procedure (Tolerance < NumberOfSimulations)
*ABCTolerance=50
```

- **ABC method.** The ABC algorithm must be specified. The user can choose either to perform only the rejection algorithm, by choosing the option "rejection", or to perform both the rejection and the multiple linear regression algorithms, by choosing the option "loclinear". For example,

```
#ABC method (rejection, loclinear)
*ABCMethod=loclinear
```

- **ABC transformation.** This parameter allows the user to choose which, if any, transformation will be performed to the data before adjusting the values with the regression algorithm. The user can choose not to perform any transformation (option "none", to perform a logarithm transformation (option "log"), or to perform a logit transformation (option "logit"). As a sanity check, even if choosing to perform only the rejection algorithm the user must specify this parameter (although it will not be used). For example,

```
#ABC transformation (none, log, logit)
*ABCTransf=logit
```

- **ABC correction.** This parameter allows the user to correct for the heteroscedasticity of the data when performing the regression adjustment. The user chooses to perform or not this correction by specifying option "1" or "0", respectively. For example,

```
#ABC correction. Correct for heteroscedastic variance (no=0, yes=1)
*ABCHCorr=1
```

Graphical options

- **Multiple pages.** This setting specifies if the plots produced by *ProteinEvolverABC* estimation phase will be presented in single-page PDF documents (option "0") or in multiple-pages PDF documents (option "1"). The last option is advised since it reduces the number of files produced (see section on "Output Files" for details). For example,

```
#Multiple pages. PDF documents with multiple pages (no=0, yes=1)
*MultiPage=1
```

3.2. The target alignment of protein sequences

The name of the file with the multiple alignment of protein sequences to be analyzed must be indicated in the Settings.txt file (specified in *NameOfPhylipFile*). Importantly, this file must be placed in the directory of the settings file and only its filename should be specified (i.e. do not include its pathway).

The alignment must be presented in *standard phylip sequential* format. In particular, the first line must indicate the number of sequences and the sequence length (in amino acids). The following lines must indicate a taxa name (10 characters) followed by the sequence (the amino acids in one letter code or indels as "-"). A variety of examples are provided in the *ProteinEvolverABC* package (in the folder "examples").

Example,

```
5 15
seq000001 AYPFQLGFQDATSPI
seq000002 AYPLQLGFQDATSPI
seq000003 AYPFQLGFQDAASPI
seq000004 AYPLQLGFQDATSPI
seq000005 AYPLQLGFQDAVSPI
```

In order to avoid errors generating the correct format, we recommend use the format alignment converter *ALTER* (http://sing.ei.uvigo.es/ALTER/) [29]. Therein after uploading your input alignment, go to "*Select program*" and in the section "*Population genetics*" please select "*CodABC*". Then select "*PHYLIP*" and finally, covert.

3.3. The protein structure

A file with the protein structure (i.e. a PDB file) is required in case a SCS model or summary statistics based on the protein structures is specified. The name of the file with the protein structure must be specified in the settings file (under the *pdb* and/or *SSpdb* parameter). This file must be in the same folder as the settings file. Use only the filename (e.g., 8C0G.pdb) without the full path. It is highly recommended to select a representative structure without no insertions and deletions (gaps), that completely aligns with the sequences of the multiple sequence alignment, thus providing structural information for each protein site. Additionally, a chain of that protein structure can be indicated the settings file (under the *chain* and/or *SSchain* parameter).

3.4. The sequence of the protein structure

The sequence of the protein structure is required in case a SCS model or summary statistics based on the protein structures is specified. In case of the SCS model, it is considered in the simulations by its assigning to the GMRCA (root) node [3]. Indeed, it is required to calculate summary statistics related to the protein structure, such as the protein folding stability through the tool *Prot_evol* [8, 9]. The sequence of the protein structure must be included in a file (i.e., seqGMRCA) without any particular format (just the amino acid sequence), which must be, as the previous input files, placed in the same folder as the settings file. Indeed, the name of this file must be indicated the settings file (under the *pdbsequence* and/or *SSpdbsequence* parameter).

3.5. Screen information

The information written on the screen during an execution example is shown below.

-> At the **parent directory** of the *ProteinEvolverABC* package, we copied there the settings and multiple sequence alignment files of the example "Example1_JTT_20SS_rapid" provided with the package, which is an example based on only 500 simulations under the empirical JTT substitution model of protein evolution and considering all the implemented summary statistics, including those based on constraints from the protein structure,

```
Example1_JTT_20SS_rapid % ls
8C0G.pdb     Makefile     ProteinEvolverABC_Phase3.pl seqGMRCA   bin
       MSA_06460.phy       ProteinEvolverABC_Phase4.pl      Settings.txt
       documentation       ProteinEvolverABC_Phase1.pl
       ProteinEvolverABC_Phase5.pl      source       licenseGPL
       ProteinEvolverABC_Phase2.pl      scripts
```

-> **Compile *ProteinEvolverABC*** (it will compile *Phi*, *ProteinEvolverProtABC, ProtASR2* and SSCPE). First, we can clean it to remove previous executable files. Next, we can compile it, which can take from seconds to a minute. Some minor warnings, depending on the used system, can appear and can be ignored,

```
Example1_JTT_20SS_rapid % make clean
Removing executables ..
/Library/Developer/CommandLineTools/usr/bin/make -C source/PhiPack/src clean
rm -f *.o
rm -f Phi
rm -f Profile
/Library/Developer/CommandLineTools/usr/bin/make   -C      source/ProteinEvolverProtABC
clean
Removing object and executable files to save space
Finished cleanup.

/Library/Developer/CommandLineTools/usr/bin/make   -C     source/Prot_evol_ProtASR2_src
clean
rm  -fr  Prot evol.o  REM.o  meanfield.o  meta meanfield.o  random3.o  Fit mutation.o
Subst_exhaustive.o read_ali.o Get_pars_pop_dyn.o Codes.o Input.o gen_code.o mutation.o
allocate.o read_pdb.o read.o Sec_str_all.o read_str_mut.o Print_meanfield.o output.o
fits.o optimization.o Prot evol
Done!


Example1_JTT_20SS_rapid % make all
Compiling Phi ..
/Library/Developer/CommandLineTools/usr/bin/make -C source/PhiPack/src clean
rm -f *.o
rm -f Phi
rm -f Profile
/Library/Developer/CommandLineTools/usr/bin/make -C source/PhiPack/src Phi
gcc -c  -O3  -Wall normal.c -o normal.o
gcc -c  -O3  -Wall stats.c -o stats.o
stats.c:97:15: warning: variable 'states i' set but not used [-Wunused-but-set-variable]
   97 |   int p_i,p_j,states_i,states_j;
      |               ^
stats.c:97:24: warning: variable 'states_j' set but not used [-Wunused-but-set-variable]
   97 |   int p i,p j,states i,states j;
      |                        ^
2 warnings generated.
gcc -c  -O3  -Wall maxChi.c -o maxChi.o
gcc -c  -O3  -Wall main.c -o main.o
In file included from main.c:17:
./normal.h:1:9: warning: 'NORM' is used as a header guard here, followed by #define of a
different macro [-Wheader-guard]
    1 | #ifndef NORM
      |         ^~~~
./normal.h:2:9: note: 'NORMAL' is defined here; did you mean 'NORM'?
    2 | #define NORMAL
      |         ^~~~~~
      |         NORM
1 warning generated.
gcc -c  -O3  -Wall queue.c -o queue.o
gcc -c  -O3  -Wall graphCode.c -o graphCode.o
gcc -c  -O3  -Wall fasta.c -o fasta.o
gcc -c  -O3  -Wall phylip.c -o phylip.o
gcc -c  -O3  -Wall mem.c -o mem.o
gcc -c  -O3  -Wall misc.c -o misc.o
gcc -c  -O3  -Wall pairScore.c -o pairScore.o
gcc -c  -O3  -Wall seqManip.c -o seqManip.o
gcc -c  -O3  -Wall global.c -o global.o
gcc -O3  -Wall  -o Phi normal.o stats.o maxChi.o main.o queue.o graphCode.o fasta.o
phylip.o mem.o misc.o pairScore.o seqManip.o global.o  -lm
cp Phi ../
cd ppma_2_bmp && /Library/Developer/CommandLineTools/usr/bin/make
make[2]: Nothing to be done for `default'.
cp ppma 2 bmp/ppma 2 bmp ../
Done!

Compiling ProteinEvolverProtABC ..
/Library/Developer/CommandLineTools/usr/bin/make -C source/ProteinEvolverProtABC clean
Removing object and executable files to save space
```

```
Finished cleanup.

/Library/Developer/CommandLineTools/usr/bin/make -C source/ProteinEvolverProtABC all
Building ProteinEvolverProtABC version 2.0.0
gcc -c -w -O3 -Wall ProteinEvolverProtABC2.0.0.c
gcc  -lm -w -O3 -Wall -o ProteinEvolverProtABC2.0.0 ProteinEvolverProtABC2.0.0.o
Finished compiling.

Done!
Compiling SSCPE ..
Archive:  source/SSCPE-main.zip
   creating: ./source//SSCPE-main
  inflating: ./source//SSCPE-main/Input_TNM.in
 extracting: ./source//SSCPE-main/SSCPE.zip
 extracting: ./source//SSCPE-main/DE.zip
  inflating: ./source//SSCPE-main/README.md
 extracting: ./source//SSCPE-main/Alignments.zip
 extracting: ./source//SSCPE-main/RMSD.zip
Archive:  source/SSCPE-main/SSCPE.zip
   creating: ./source/SSCPE-main//ALI
  inflating: ./source/SSCPE-main//Input_Prot_evol.in
  inflating: ./source/SSCPE-main//Input_TNM.in
  inflating: ./source/SSCPE-main//jtt.txt
  inflating: ./source/SSCPE-main//k2_mat.pl
  inflating: ./source/SSCPE-main//lg.txt
  inflating: ./source/SSCPE-main//Mutation para.in
   creating: ./source/SSCPE-main//PDB
 extracting: ./source/SSCPE-main//Prot_evol.zip
  inflating: ./source/SSCPE-main//qsubmit.pl
  inflating: ./source/SSCPE-main//raxml-ng
  inflating: ./source/SSCPE-main//README SSCPE
  inflating: ./source/SSCPE-main//script dir Prot evol.pl
  inflating: ./source/SSCPE-main//script_install_SSCPE.sh
  inflating: ./source/SSCPE-main//script_read_tree.pl
  inflating: ./source/SSCPE-main//script_run_models.pl
  inflating: ./source/SSCPE-main//script_run_SSCPE_models.pl
  inflating: ./source/SSCPE-main//script_write_dir.pl
  inflating: ./source/SSCPE-main//SSCPE.pl
  inflating: ./source/SSCPE-main//structures.in
 extracting: ./source/SSCPE-main//tnm.zip
  inflating: ./source/SSCPE-main//wag.txt
Archive:  Alignments.zip
caution: filename not matched:  DE.zip
caution: filename not matched:  Prot_evol.zip
caution: filename not matched:  RMSD.zip
caution: filename not matched:  SSCPE.zip
caution: filename not matched:  tnm.zip
Directory where SSCPE.pl is installed:
/Users/miguelarenasbusto/Desktop/Example1_JTT_20SS_rapid
Compiling tnm in folder DIR_TNM and moving to
/Users/miguelarenasbusto/Desktop/Example1_JTT_20SS_rapid
Archive:  tnm.zip
  inflating: align_tnm.c
  inflating: align tnm.h
  inflating: align tnm old.c
  inflating: align_tnm_old.h
  inflating: allocate.c
  inflating: allocate.h
  inflating: allocate tnm.c
 extracting: allocate tnm.h
  inflating: allostery.c
  inflating: allostery.h
  inflating: allostery_old.c
  inflating: anharmonic_tnm.c
  inflating: anharmonic_tnm.h
  inflating: atom_numb.h
  inflating: atoms.c
  inflating: Atoms_distances.h
  inflating: Atoms_force_constant.h
  inflating: atoms.h
  inflating: Atoms propensity.h
 extracting: Bfactors_fit.c
 extracting: binding_free_energy.c
  inflating: binding site.c
  inflating: binding sites.h
  inflating: bonds.h
  inflating: buildup.c
```

```
  inflating: buildup.h
  inflating: buildup_new.c
  inflating: buildup_old.c
  inflating: choldc.c
  inflating: choldc.h
  inflating: confchange.c
  inflating: confchange_old.c
  inflating: contacts.c
  inflating: contacts.h
  inflating: coord.h
  inflating: d diagonalize.c
  inflating: d_diagonalize_old.c
  inflating: diagonalize.h
  inflating: direct interactions.h
  inflating: direct interactions Ugo.c
  inflating: dof_tnm.c
  inflating: dof_tnm.h
  inflating: dof_tnm_new.c
  inflating: dof_tnm_old.c
  inflating: EC.c
  inflating: EC.h
  inflating: energy_BKV.h
  inflating: externals.h
  inflating: Fit_B_factors.c
  inflating: Fit_B.h
  inflating: Fit.c
  inflating: force2confchange.c
  inflating: force2confchange.h
  inflating: gasdev.c
  inflating: Input TNM.in
  inflating: Input TNM mutation.in
  inflating: interactions tnm.c
  inflating: interactions_tnm.h
  inflating: interactions_tnm_old.c
  inflating: jacobi.c
  inflating: kinetic_tnm.c
  inflating: kinetic_tnm.h
  inflating: kinetic tnm new.c
  inflating: kinetic tnm old.c
  inflating: List_test_all.txt
  inflating: lubksb.c
  inflating: ludcmp.c
  inflating: macros.h
  inflating: main_standard_bonds.c
  inflating: main_tnm.c
  inflating: main_tnm_old.c
  inflating: Makefile
  inflating: mammoth_interface.c
  inflating: mammoth_interface.h
  inflating: McLachlan.c
  inflating: McLachlan_float.c
  inflating: McLachlan.h
  inflating: mutation.c
  inflating: mutation.h
  inflating: Mutation para.in
  inflating: NeedlemanWunsch.c
  inflating: NeedlemanWunsch.h
  inflating: nma.c
  inflating: nma.h
  inflating: nma old.c
  inflating: nma_old.h
  inflating: nma_para.h
  inflating: nrutil.c
  inflating: nrutil.h
  inflating: optimization.c
  inflating: optimization.h
  inflating: output_tnm.c
  inflating: output_tnm.h
  inflating: Profit_aux.c
  inflating: Profit aux.h
  inflating: pythag.c
  inflating: ran2.c
  inflating: random3.c
  inflating: random3.h
  inflating: read.c
  inflating: read.h
  inflating: README
```

```
  inflating: readme.h
  inflating: Residues_distances.h
  inflating: Residues_force_constant.h
  inflating: Residues_propensity.h
  inflating: ridge_regression.c
  inflating: ridge_regression.h
  inflating: ridge_regression_old.c
 extracting: rotamers.h
  inflating: screened_interactions.c
  inflating: script read binding.pl
  inflating: script run tnm.pl
  inflating: seqres.c
  inflating: shadow_interactions.c
  inflating: simulation.c
  inflating: simulation.h
  inflating: simulation_old.c
  inflating: site_dynamics.c
  inflating: tnm.h
  inflating: tqli.c
  inflating: tred2.c
  inflating: unfolding.c
  inflating: unfolding.h
  inflating: vector.c
  inflating: vector.h
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o main_tnm.o main_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o read.o read.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o atoms.o atoms.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o seqres.o seqres.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o vector.o vector.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o allocate_tnm.o allocate_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o align_tnm.o align_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o dof_tnm.o dof_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o buildup.o buildup.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o interactions_tnm.o interactions_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o anharmonic_tnm.o anharmonic_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o contacts.o contacts.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o screened_interactions.o screened_interactions.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o kinetic_tnm.o kinetic_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o nma.o nma.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o Fit_B_factors.o Fit_B_factors.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o ridge_regression.o ridge_regression.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o confchange.o confchange.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o simulation.o simulation.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o force2confchange.o force2confchange.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o mutation.o mutation.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o Fit.o Fit.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o optimization.o optimization.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o output_tnm.o output_tnm.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o allostery.o allostery.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o site_dynamics.o site_dynamics.c
```

```
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o allocate.o allocate.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o choldc.o choldc.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o ludcmp.o ludcmp.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o lubksb.o lubksb.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D LARGE FILE SOURCE   -c -o d diagonalize.o d diagonalize.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D FILE OFFSET BITS=64   -
D_LARGE_FILE_SOURCE   -c -o nrutil.o nrutil.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D LARGE FILE SOURCE   -c -o pythag.o pythag.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D FILE OFFSET BITS=64   -
D_LARGE_FILE_SOURCE   -c -o tred2.o tred2.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o jacobi.o jacobi.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o tqli.o tqli.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o Profit_aux.o Profit_aux.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o NeedlemanWunsch.o NeedlemanWunsch.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D LARGE FILE SOURCE   -c -o McLachlan float.o McLachlan float.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D FILE OFFSET BITS=64   -
D_LARGE_FILE_SOURCE   -c -o McLachlan.o McLachlan.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D LARGE FILE SOURCE   -c -o EC.o EC.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D FILE OFFSET BITS=64   -
D LARGE FILE SOURCE   -c -o gasdev.o gasdev.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o ran2.o ran2.c
gcc   -g   -O2   -Wall   -w   -std=c99   -pg   -fbounds-check   -D_FILE_OFFSET_BITS=64   -
D_LARGE_FILE_SOURCE   -c -o random3.o random3.c
gcc   main_tnm.o read.o atoms.o seqres.o vector.o allocate_tnm.o align_tnm.o dof_tnm.o
buildup.o   interactions tnm.o   anharmonic tnm.o   contacts.o   screened interactions.o
kinetic tnm.o   nma.o   Fit B factors.o   ridge regression.o   confchange.o   simulation.o
force2confchange.o   mutation.o   Fit.o   optimization.o   output_tnm.o   allostery.o
site_dynamics.o allocate.o choldc.o ludcmp.o lubksb.o d_diagonalize.o nrutil.o pythag.o
tred2.o  jacobi.o  tqli.o  Profit aux.o  NeedlemanWunsch.o  McLachlan float.o  McLachlan.o
EC.o gasdev.o ran2.o random3.o   -o tnm -lm -L/usr/lib64/libg2c.so.0.0.0
ld: warning: search path '/usr/lib64/libg2c.so.0.0.0' not found
Compiling Prot_evol in folder DIR_PROT_EVOL and moving to current folder
Archive:  Prot_evol.zip
  inflating: makefile
  inflating: README_Prot_evol
  inflating: Prot_evol_mean_field.c
  inflating: REM.c
  inflating: meanfield.c
  inflating: meta_meanfield.c
  inflating: random3.c
  inflating: Fit mutation.c
  inflating: Subst exhaustive.c
  inflating: Get_pars_pop_dyn.c
  inflating: Codes.c
  inflating: Input.c
  inflating: gen code.c
  inflating: mutation.c
  inflating: allocate.c
  inflating: read_pdb.c
  inflating: read.c
  inflating: Sec_str_all.c
  inflating: Print_meanfield.c
  inflating: output.c
  inflating: fits.c
  inflating: optimization.c
  inflating: aa_freq_UniProt.h
  inflating: alignments.h
  inflating: allocate.h
  inflating: codes.h
  inflating: contact_matrix.h
  inflating: coord.h
  inflating: dna evol aux.h
  inflating: energy BKV.h
  inflating: fits.h
```

```
  inflating: gen_code.h
  inflating: get_para_DeltaGREM.h
  inflating: get_para_pop_dyn.h
  inflating: get_para_pop_dyn_old.h
  inflating: input.h
  inflating: jtt.h
 extracting: load.h
  inflating: meanfield.h
  inflating: mutation.h
  inflating: mut_del.h
  inflating: NeedlemanWunsch.h
  inflating: optimization.h
  inflating: output.h
  inflating: Print_meanfield.h
  inflating: Profit_aux.h
  inflating: protein3.h
 extracting: random0.h
  inflating: random3.h
  inflating: read_pdb.h
  inflating: REM.h
  inflating: Sec_str_16_NR.h
  inflating: Sec_str_all.h
  inflating: Sec_str_comp.h
  inflating: selection.h
  inflating: subst_exhaustive.h
  inflating: thread_contfreq.h
  inflating: thread_rem.h
  inflating: wag.h
  inflating: Input_Prot_evol.in
  inflating: script_run_Prot_Evol.pl
  inflating: Get_pars_DeltaGREM.c
  inflating: main_DeltaGREM.c
  inflating: mutations.c
  inflating: Prot_evol.c
  inflating: alignments.c
  inflating: Analyze_exchange.c
  inflating: contact_matrix.c
  inflating: d_diagonalize.c
  inflating: Get_pars_pop_dyn_old.c
  inflating: main_wagmut.c
  inflating: meanfield_old.c
  inflating: NeedlemanWunsch.c
  inflating: nrutil.c
  inflating: Print_meanfield_old.c
  inflating: Profit_aux.c
  inflating: Prot_evol_simul_old.c
  inflating: read_ali.c
  inflating: read_str_mut.c
  inflating: REM_old.c
  inflating: selection.c
  inflating: selection_old.c
  inflating: thread_contfreq.c
  inflating: tqli.c
  inflating: tred2.c
  inflating: diagonalize.h
  inflating: nrutil.h
  inflating: read_ali.h
  inflating: read_str_mut.h
  inflating: structures.in
  inflating: externals.h
  inflating: Pairwise_aux.c
  inflating: read_pdb_old.c
  inflating: make_DeltaGREM
  inflating: Input_DeltaGREM.in
  inflating: make_DeltaGREM.makefile
 extracting: REM_new.h
  inflating: REM_old.h
  inflating: lg.h
Working                                                                  directory:
/Users/miguelarenasbusto/Desktop/Example1_JTT_20SS_rapid/DIR_PROT_EVOL
Installation directory: /Users/miguelarenasbusto/Desktop/Example1_JTT_20SS_rapid
Prot_evol.c was modified at 4 lines
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Prot_evol.o Prot_evol.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
REM.o REM.c
```

```
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
meanfield.o meanfield.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
meta_meanfield.o meta_meanfield.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
random3.o random3.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Fit_mutation.o Fit_mutation.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Subst exhaustive.o Subst exhaustive.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
read_ali.o read_ali.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
NeedlemanWunsch.o NeedlemanWunsch.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
Profit_aux.o Profit_aux.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Get_pars_pop_dyn.o Get_pars_pop_dyn.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Codes.o Codes.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Input.o Input.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
gen_code.o gen_code.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
mutation.o mutation.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
allocate.o allocate.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
read pdb.o read pdb.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
read.o read.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Sec_str_all.o Sec_str_all.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
read_str_mut.o read_str_mut.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Print meanfield.o Print meanfield.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
output.o output.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
fits.o fits.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
optimization.o optimization.c
gcc   Prot_evol.o   REM.o   meanfield.o   meta_meanfield.o   random3.o   Fit_mutation.o
Subst_exhaustive.o read_ali.o NeedlemanWunsch.o Profit_aux.o Get_pars_pop_dyn.o Codes.o
Input.o gen_code.o mutation.o allocate.o read_pdb.o read.o Sec_str_all.o read_str_mut.o
Print_meanfield.o    output.o    fits.o    optimization.o    -o    Prot_evol    -lm    -
L/usr/lib64/libg2c.so.0.0.0
ld: warning: search path '/usr/lib64/libg2c.so.0.0.0' not found
Assigning dependencies of SSCPE.pl
Running the program script_write_dir.pl
Working directory: /Users/miguelarenasbusto/Desktop/Example1_JTT_20SS_rapid
SSCPE.pl was modified
script run SSCPE models.pl was modified
Input_TNM.in was modified
Input_Prot_evol.in was modified
All done
Executable file SSCPE.pl is in folder:
/Users/miguelarenasbusto/Desktop/Example1 JTT 20SS rapid
Done!
Compiling Prot_evol3 ..
Archive:  source/SSCPE-main.zip
   creating: ./source//SSCPE-main
  inflating: ./source//SSCPE-main/Input_TNM.in
 extracting: ./source//SSCPE-main/SSCPE.zip
 extracting: ./source//SSCPE-main/DE.zip
  inflating: ./source//SSCPE-main/README.md
 extracting: ./source//SSCPE-main/Alignments.zip
 extracting: ./source//SSCPE-main/RMSD.zip
Archive:  source/SSCPE-main/SSCPE.zip
   creating: ./source/SSCPE-main//ALI
  inflating: ./source/SSCPE-main//Input_Prot_evol.in
  inflating: ./source/SSCPE-main//Input TNM.in
  inflating: ./source/SSCPE-main//jtt.txt
  inflating: ./source/SSCPE-main//k2 mat.pl
  inflating: ./source/SSCPE-main//lg.txt
```

```
  inflating: ./source/SSCPE-main//Mutation_para.in
   creating: ./source/SSCPE-main//PDB
 extracting: ./source/SSCPE-main//Prot_evol.zip
  inflating: ./source/SSCPE-main//qsubmit.pl
  inflating: ./source/SSCPE-main//raxml-ng
  inflating: ./source/SSCPE-main//README_SSCPE
  inflating: ./source/SSCPE-main//script_dir_Prot_evol.pl
  inflating: ./source/SSCPE-main//script_install_SSCPE.sh
  inflating: ./source/SSCPE-main//script_read_tree.pl
  inflating: ./source/SSCPE-main//script_run_models.pl
  inflating: ./source/SSCPE-main//script_run_SSCPE_models.pl
  inflating: ./source/SSCPE-main//script_write_dir.pl
  inflating: ./source/SSCPE-main//SSCPE.pl
  inflating: ./source/SSCPE-main//structures.in
 extracting: ./source/SSCPE-main//tnm.zip
  inflating: ./source/SSCPE-main//wag.txt
Archive:  source/SSCPE-main/Prot_evol.zip
  inflating: ./source/SSCPE-main/Prot_evol3//makefile
  inflating: ./source/SSCPE-main/Prot_evol3//README_Prot_evol
  inflating: ./source/SSCPE-main/Prot_evol3//Prot_evol_mean_field.c
  inflating: ./source/SSCPE-main/Prot_evol3//REM.c
  inflating: ./source/SSCPE-main/Prot_evol3//meanfield.c
  inflating: ./source/SSCPE-main/Prot_evol3//meta_meanfield.c
  inflating: ./source/SSCPE-main/Prot_evol3//random3.c
  inflating: ./source/SSCPE-main/Prot_evol3//Fit_mutation.c
  inflating: ./source/SSCPE-main/Prot_evol3//Subst_exhaustive.c
  inflating: ./source/SSCPE-main/Prot_evol3//Get_pars_pop_dyn.c
  inflating: ./source/SSCPE-main/Prot_evol3//Codes.c
  inflating: ./source/SSCPE-main/Prot_evol3//Input.c
  inflating: ./source/SSCPE-main/Prot_evol3//gen_code.c
  inflating: ./source/SSCPE-main/Prot_evol3//mutation.c
  inflating: ./source/SSCPE-main/Prot_evol3//allocate.c
  inflating: ./source/SSCPE-main/Prot_evol3//read_pdb.c
  inflating: ./source/SSCPE-main/Prot_evol3//read.c
  inflating: ./source/SSCPE-main/Prot_evol3//Sec_str_all.c
  inflating: ./source/SSCPE-main/Prot_evol3//Print_meanfield.c
  inflating: ./source/SSCPE-main/Prot_evol3//output.c
  inflating: ./source/SSCPE-main/Prot_evol3//fits.c
  inflating: ./source/SSCPE-main/Prot_evol3//optimization.c
  inflating: ./source/SSCPE-main/Prot_evol3//aa_freq_UniProt.h
  inflating: ./source/SSCPE-main/Prot_evol3//alignments.h
  inflating: ./source/SSCPE-main/Prot_evol3//allocate.h
  inflating: ./source/SSCPE-main/Prot_evol3//codes.h
  inflating: ./source/SSCPE-main/Prot_evol3//contact_matrix.h
  inflating: ./source/SSCPE-main/Prot_evol3//coord.h
  inflating: ./source/SSCPE-main/Prot_evol3//dna_evol_aux.h
  inflating: ./source/SSCPE-main/Prot_evol3//energy_BKV.h
  inflating: ./source/SSCPE-main/Prot_evol3//fits.h
  inflating: ./source/SSCPE-main/Prot_evol3//gen_code.h
  inflating: ./source/SSCPE-main/Prot_evol3//get_para_DeltaGREM.h
  inflating: ./source/SSCPE-main/Prot_evol3//get_para_pop_dyn.h
  inflating: ./source/SSCPE-main/Prot_evol3//get_para_pop_dyn_old.h
  inflating: ./source/SSCPE-main/Prot_evol3//input.h
  inflating: ./source/SSCPE-main/Prot_evol3//jtt.h
 extracting: ./source/SSCPE-main/Prot_evol3//load.h
  inflating: ./source/SSCPE-main/Prot_evol3//meanfield.h
  inflating: ./source/SSCPE-main/Prot_evol3//mutation.h
  inflating: ./source/SSCPE-main/Prot_evol3//mut_del.h
  inflating: ./source/SSCPE-main/Prot_evol3//NeedlemanWunsch.h
  inflating: ./source/SSCPE-main/Prot_evol3//optimization.h
  inflating: ./source/SSCPE-main/Prot_evol3//output.h
  inflating: ./source/SSCPE-main/Prot_evol3//Print_meanfield.h
  inflating: ./source/SSCPE-main/Prot_evol3//Profit_aux.h
  inflating: ./source/SSCPE-main/Prot_evol3//protein3.h
 extracting: ./source/SSCPE-main/Prot_evol3//random0.h
  inflating: ./source/SSCPE-main/Prot_evol3//random3.h
  inflating: ./source/SSCPE-main/Prot_evol3//read_pdb.h
  inflating: ./source/SSCPE-main/Prot_evol3//REM.h
  inflating: ./source/SSCPE-main/Prot_evol3//Sec_str_16_NR.h
  inflating: ./source/SSCPE-main/Prot_evol3//Sec_str_all.h
  inflating: ./source/SSCPE-main/Prot_evol3//Sec_str_comp.h
  inflating: ./source/SSCPE-main/Prot_evol3//selection.h
  inflating: ./source/SSCPE-main/Prot_evol3//subst_exhaustive.h
  inflating: ./source/SSCPE-main/Prot_evol3//thread_contfreq.h
  inflating: ./source/SSCPE-main/Prot_evol3//thread_rem.h
  inflating: ./source/SSCPE-main/Prot_evol3//wag.h
  inflating: ./source/SSCPE-main/Prot_evol3//Input_Prot_evol.in
```

```
  inflating: ./source/SSCPE-main/Prot_evol3//script_run_Prot_Evol.pl
  inflating: ./source/SSCPE-main/Prot_evol3//Get_pars_DeltaGREM.c
  inflating: ./source/SSCPE-main/Prot_evol3//main_DeltaGREM.c
  inflating: ./source/SSCPE-main/Prot_evol3//mutations.c
  inflating: ./source/SSCPE-main/Prot_evol3//Prot_evol.c
  inflating: ./source/SSCPE-main/Prot_evol3//alignments.c
  inflating: ./source/SSCPE-main/Prot_evol3//Analyze_exchange.c
  inflating: ./source/SSCPE-main/Prot_evol3//contact_matrix.c
  inflating: ./source/SSCPE-main/Prot_evol3//d_diagonalize.c
  inflating: ./source/SSCPE-main/Prot_evol3//Get_pars_pop_dyn_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//main_wagmut.c
  inflating: ./source/SSCPE-main/Prot_evol3//meanfield_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//NeedlemanWunsch.c
  inflating: ./source/SSCPE-main/Prot_evol3//nrutil.c
  inflating: ./source/SSCPE-main/Prot_evol3//Print_meanfield_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//Profit_aux.c
  inflating: ./source/SSCPE-main/Prot_evol3//Prot_evol_simul_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//read_ali.c
  inflating: ./source/SSCPE-main/Prot_evol3//read_str_mut.c
  inflating: ./source/SSCPE-main/Prot_evol3//REM_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//selection.c
  inflating: ./source/SSCPE-main/Prot_evol3//selection_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//thread_contfreq.c
  inflating: ./source/SSCPE-main/Prot_evol3//tqli.c
  inflating: ./source/SSCPE-main/Prot_evol3//tred2.c
  inflating: ./source/SSCPE-main/Prot_evol3//diagonalize.h
  inflating: ./source/SSCPE-main/Prot_evol3//nrutil.h
  inflating: ./source/SSCPE-main/Prot_evol3//read_ali.h
  inflating: ./source/SSCPE-main/Prot_evol3//read_str_mut.h
  inflating: ./source/SSCPE-main/Prot_evol3//structures.in
  inflating: ./source/SSCPE-main/Prot_evol3//externals.h
  inflating: ./source/SSCPE-main/Prot_evol3//Pairwise_aux.c
  inflating: ./source/SSCPE-main/Prot_evol3//read_pdb_old.c
  inflating: ./source/SSCPE-main/Prot_evol3//make_DeltaGREM
  inflating: ./source/SSCPE-main/Prot_evol3//Input_DeltaGREM.in
  inflating: ./source/SSCPE-main/Prot_evol3//make_DeltaGREM.makefile
 extracting: ./source/SSCPE-main/Prot_evol3//REM_new.h
  inflating: ./source/SSCPE-main/Prot_evol3//REM_old.h
  inflating: ./source/SSCPE-main/Prot_evol3//lg.h

Paths Prot_evol3 done!
/Library/Developer/CommandLineTools/usr/bin/make -C source/SSCPE-main/Prot_evol3
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Prot_evol.o Prot_evol.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
REM.o REM.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
meanfield.o meanfield.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
meta_meanfield.o meta_meanfield.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
random3.o random3.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Fit_mutation.o Fit_mutation.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Subst_exhaustive.o Subst_exhaustive.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
read_ali.o read_ali.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
NeedlemanWunsch.o NeedlemanWunsch.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Profit_aux.o Profit_aux.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Get_pars_pop_dyn.o Get_pars_pop_dyn.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Codes.o Codes.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Input.o Input.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
gen_code.o gen_code.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
mutation.o mutation.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
allocate.o allocate.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
read_pdb.o read_pdb.c
```

```
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
read.o read.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Sec_str_all.o Sec_str_all.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
read_str_mut.o read_str_mut.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
Print_meanfield.o Print_meanfield.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
output.o output.c
gcc -O2 -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE    -c -o
fits.o fits.c
gcc -O2 -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE    -c -o
optimization.o optimization.c
gcc   Prot evol.o   REM.o   meanfield.o   meta meanfield.o   random3.o   Fit mutation.o
Subst_exhaustive.o read_ali.o NeedlemanWunsch.o Profit_aux.o Get_pars_pop_dyn.o Codes.o
Input.o gen_code.o mutation.o allocate.o read_pdb.o read.o Sec_str_all.o read_str_mut.o
Print_meanfield.o    output.o    fits.o    optimization.o    -o    Prot_evol    -lm    -
L/usr/lib64/libg2c.so.0.0.0
ld: warning: search path '/usr/lib64/libg2c.so.0.0.0' not found
Done!
Compiling Prot_evol_ProtASR2 ..
/Library/Developer/CommandLineTools/usr/bin/make -C source/Prot_evol_ProtASR2_src
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
Prot_evol.o Prot_evol.c
gcc -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE   -c -o REM.o
REM.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
meanfield.o meanfield.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D FILE OFFSET BITS=64  -D LARGE FILE SOURCE     -c  -o
meta meanfield.o meta meanfield.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D FILE OFFSET BITS=64  -D LARGE FILE SOURCE     -c  -o
random3.o random3.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
Fit_mutation.o Fit_mutation.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
Subst_exhaustive.o Subst_exhaustive.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D FILE OFFSET BITS=64  -D LARGE FILE SOURCE     -c  -o
read ali.o read ali.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
Get_pars_pop_dyn.o Get_pars_pop_dyn.c
gcc -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE   -c -o Codes.o
Codes.c
gcc -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE   -c -o Input.o
Input.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
gen_code.o gen_code.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
mutation.o mutation.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
allocate.o allocate.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
read_pdb.o read_pdb.c
gcc -Wall -w -std=c99 -g -pg -D FILE OFFSET BITS=64 -D LARGE FILE SOURCE   -c -o read.o
read.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
Sec_str_all.o Sec_str_all.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D FILE OFFSET BITS=64  -D LARGE FILE SOURCE     -c  -o
read str mut.o read str mut.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D FILE OFFSET BITS=64  -D LARGE FILE SOURCE     -c  -o
Print_meanfield.o Print_meanfield.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
output.o output.c
gcc -Wall -w -std=c99 -g -pg -D_FILE_OFFSET_BITS=64 -D_LARGE_FILE_SOURCE   -c -o fits.o
fits.c
gcc  -Wall  -w  -std=c99  -g  -pg  -D_FILE_OFFSET_BITS=64  -D_LARGE_FILE_SOURCE     -c  -o
optimization.o optimization.c
gcc   Prot_evol.o   REM.o   meanfield.o   meta_meanfield.o   random3.o   Fit_mutation.o
Subst_exhaustive.o read_ali.o Get_pars_pop_dyn.o Codes.o Input.o gen_code.o mutation.o
allocate.o read pdb.o read.o Sec str all.o read str mut.o Print meanfield.o output.o
fits.o optimization.o -o Prot evol -lm -L/usr/lib64/libg2c.so.0.0.0
ld: warning: search path '/usr/lib64/libg2c.so.0.0.0' not found
Done!

Note that R will require the following libraries: lattice, MCMCpack, ape, graphics, abc
These libraries can be installed from R by typing:
install.packages(lattice)
```

```
install.packages(MCMCpack)
install.packages(ape)
install.packages(graphics)
install.packages(abc)
See the documentation for additional details about ProteinEvolverABC
Compilation completed!
```

Notice, at the end, the reminder about the required R libraries (lattice, MCMCpack, ape, graphics, abc). These R libraries must be installed in the OS.
Executable files will be placed in the "*bin*" folder from where *ProteinEvolverABC* will use them,

```
Example1_JTT_20SS_rapid% cd bin
% ls
Phi    Prot_evol_ProtASR2_exe    ProteinEvolverProtABC2.0.0       tnm
       Prot_evol    Prot_evol3   SSCPE
```

-> Now one can run *ProteinEvolverABC* with **the settings and multiple sequence alignment files, as well as the other input files if needed, in the working directory**,

```
bin % cd ..
Example1_JTT_20SS_rapid % ls
8C0G.pdb     Makefile     ProteinEvolverABC_Phase3.pl      seqGMRCA     bin
       MSA_06460.phy      ProteinEvolverABC_Phase4.pl      Settings.txt
       documentation      ProteinEvolverABC_Phase1.pl
       ProteinEvolverABC_Phase5.pl      source       licenseGPL
       ProteinEvolverABC_Phase2.pl      scripts
```

-> Step 1 of *ProteinEvolverABC* is executed with *ProteinEvolverABC_Phase1.pl* and indicating the settings file. This phase reads the settings file, checks and saves information about the evolutionary scenario and takes samples from the user-specified prior distributions of the parameters to be estimated using R.

```
Example1_JTT_20SS_rapid % perl ProteinEvolverABC_Phase1.pl Settings.txt

*******************************************************************************
**********************************
ProteinEvolverABC_Phase1.pl
- Read Settings file
- Compute the summary statistics for the target data
- Analyze the evolutionary scenario
- Compute the prior distributions (requires R with libraries: lattice,
MCMCpack, ape, graphics)
*******************************************************************************
**********************************

> Input file uploaded: Settings.txt

> ProteinEvolverABC directory detected: .

> Settings directory detected: .

> OS detected: darwin (Mac OS X):
  darwin
  darwin-thread-multi-2level

> R output file created: "./MakePriorsProteinEvolverABC.r" ...

> Reading Settings from input main file ...
  Target alignment file: MSA_06460.phy
    ./MSA_06460.phy was detected... Ok!
  Number of simulations: 500
  Indels (gaps) are considered as a new state
  Number of processors to run the simulations: 1
```

```
  Simulated data will not be saved
  Running information (simulations and summary statistics) is not shown on the
screen
  Haploid/diploid: 2
  Population size according to: fix 1000
  Recombination rate according to: uniform 0 6.92e-5
  Substitution rate according to: uniform 0 2.5952e-4
  Substitution model: JTT
  Amino acid frequencies (1x20): fix 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
  PDB file for summary statistics: 8C0G.pdb
    ./8C0G.pdb was detected for summary statistics... Ok!  Chain for summary
statistics: A
  File with the PDB sequence for summary statistics: default

> Reading input protein sequences file ...
 Reading input file "./MSA_06460.phy"...
 MSA_06460.phy with sample size 11 and length 289 amino acids
> Writing R output file "./MakePriorsProteinEvolverABC.r" ...

> Writing executable output file for simulations
"./ProteinEvolverABC_Phase3.sh", "./ProteinEvolverABC_arguments.txt" and
"./ProteinEvolverABC_Pop_arguments.txt" ...

> Computing prior distributions ...
This phase requires the programming language R with the installed libraries:
lattice, MCMCpack, ape, graphics
An output file is created with the R outputs ...

> Successful!
```

-> Step 2 of *ProteinEvolverABC* is executed with *ProteinEvolverABC_Phase2.pl* and indicating the settings file. Here, after reading the settings file and scanning the directory environment and detecting the operative system. If an SCS model is applied (i.e., SSCPE, ProtASR2_WT or ProtASR2_MF) the program applies that model to generate the exchangeability matrix or matrices. Then, it computes the summary statistics of the target multiple sequence alignment (more information about the summary statistics of the input multiple sequence alignment can be shown on the screen if the user has previously activated the option `ShowInformationScreen=1` in the settings file). In this example an SCS model is not applied but summary statistics that consider the protein structure are calculated for the studied multiple sequence alignment.

```
Example1_JTT_20SS_rapid % perl ProteinEvolverABC_Phase2.pl Settings.txt

*****************************************************************************
***********************************
ProteinEvolverABC_Phase2.pl
- Calculation of the summary statistics of the real data
- Running the SSCPE and ProtASR2 models (if specified)
*****************************************************************************
***********************************

> Input file uploaded: Settings.txt

> ProteinEvolverABC directory detected: .

> Settings directory detected: .

> OS detected: darwin (Mac OS X):
  darwin
  darwin-thread-multi-2level

> Reading Settings from input main file ...
```

```
  Target alignment file: MSA_06460.phy
    ./MSA_06460.phy was detected... Ok!
    ./MSA_06460.phy_PDB.phy was detected... Ok!
  Number of simulations: 500
  Indels (gaps) are considered as a new state
  Number of processors to run the simulations: 1
  Simulated data will not be saved
  Running information (simulations and summary statistics) is not shown on the
screen
  Haploid/diploid: 2
  Population size according to: fix 1000
  Recombination rate according to: uniform 0 6.92e-5
  Substitution rate according to: uniform 0 2.5952e-4
  Substitution model: JTT
  Amino acid frequencies (1x20): fix 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05 0.05
  PDB file for summary statistics: 8C0G.pdb
    ./8C0G.pdb was detected for summary statistics... Ok!  Chain for summary
statistics: A
  File with the PDB sequence for summary statistics: default

> Reading input protein sequences file ...
 Reading input file "./MSA_06460.phy"...
 MSA_06460.phy with sample size 11 and length 289 amino acids

> Computing summary statistics from input protein sequences file ..

Done!
Modified alignment with PDB sequence written to: ./MSA_06460.phy_PDB.phy

    ./MSA_06460.phy_PDB.phy was detected... Ok!  Target alignment file:
MSA_06460.phy
  "./MSA_06460.phy" was detected... Ok!
  MSA_06460.phy with sample size 11 and length 289 amino acids

> Calculating summary statistics from the target data set (STR) ..
SSs dataset: "./MSA_06460.phy"
Summary statistics for the target data were successfully computed!
```

-> **Step 3 of *ProteinEvolverABC* is executed** with *ProteinEvolverABC_Phase3.pl* and the settings file. In this step, the computer simulations are performed. The amount of information about the computer simulations printed on the screen depends on the previous activation of the option *ShowInformationScreen* in the settings file (in the following example the information of computer simulations on the screen was not activated to save computer time). The simulations can run in parallel under the conditions specified by the user in the settings file. This step also prepares files for the calculation of summary statistics from the simulated data that is done in the next step.

```
Example1_JTT_20SS_rapid % perl ProteinEvolverABC_Phase3.pl Settings.txt

*****************************************************************************
*********************************
ProteinEvolverABC_Phase3.pl
ProteinEvolverABC Phase 3 does:
- Read Settings file
- Perform simulations
*****************************************************************************
*********************************

> OS detected: darwin (Mac OS X):
  darwin
  darwin-thread-multi-2level
```

```
> Settings directory detected: .

  Number of simulations: 500
  Indels (gaps) are considered as a new state
  Number of processors to run the simulations: 1
  Simulated data will not be saved
  Running information (simulations and summary statistics) is not shown on the
screen

  PDB file for summary statistics: 8C0G.pdb
    ./8C0G.pdb was detected for summary statistics... Ok!  Chain for summary
statistics: A
  File with the PDB sequence for summary statistics: seqGMRCA
> Performing simulations..
  "./ProteinEvolverABC_Phase3.sh" was detected... Ok!
  "./ProteinEvolverABC_arguments.txt" was detected... Ok!
Simulation # 500

> Obtaining simulations with sequence of the PDB file (given by the user) ...
PDB from seqGMRCA succesfully extracted:
'SQAWQPGVAMPNLYKMQRMLLEKCDLQNYGENAVIPKGIMMNVAKYTQLCQYLNTLTLAVPYNMRVIHFGAGSDKGV
APGTAVLRQWLPTGTLLVDSDLNDFVSDADSTLIGDCATVHTANKWDLIISDMYDPRTKHVTKENDSKEGFFTYLCGF
IKQKLALGGSIAVKITEHSWNADLYKLMGHFSWWTAFVTNVNASSSEAFLIGANYLGKPKEQIDGYTMHANYIFWRNT
NPIQLSSYSLFDMSKFPLKLRGTAVMSLKENQINDMIYSLLEKGRLIIRENNRVVV'
All done!


> Generating files "ProteinEvolverABC_Phase4.sh" and "ArgumentsSS.txt" for the
phase 4...

> Successful!
```

This step can take a considerable amount of time depending on the number of simulations to perform, the size of the simulated alignments (number of sequences and sequence length), and the recombination rate. Increasing the value of those parameters increases the computer time. Also, the applied substitution model can affect the required computer time. In particular, SCS models obviously require more time than the traditional empirical substitution models that ignore the protein structure [11, 30]. Running simulations in parallel, when possible, can reduce the computer time.
Also, when the representative structure is used and leads to extremely low stability predictions or poor alignment between the PDB sequence and the input multiple sequence alignment, there may be problems and the representative structure should be revised or changed.
At the end of this step many simulated will be present in the working directory for the next step, it should not be removed, just run the next step.

-> Step 4 of *ProteinEvolverABC* is executed with *ProteinEvolverABC_Phase4.pl* and indicating the settings file. The amount of information about the computed summary statistics that is printed on the screen depends on the previous activation of the option `ShowInformationScreen` in the settings file (in the following example the information of computer simulations on the screen was not activated to save computer time). This script computes summary statistics from the previously simulated multiple alignments of protein sequences. This step can take time, especially if it includes summary statistics that consider constraints from the protein structure.

```
Example1_JTT_20SS_rapid % perl ProteinEvolverABC_Phase4.pl Settings.txt

*******************************************************************************
**********************************
ProteinEvolverABC_Phase4.pl
```

```
ProteinEvolverABC Phase 4 does:
- Compute the summary statistics for the simulated data
*******************************************************************************
**********************************

 Indels (gaps) are considered as a new state
 Simulated data will not be saved
 Running information (simulations and summary statistics) is not shown on the
screen

  PDB file for summary statistics: 8C0G.pdb
    ./8C0G.pdb was detected for summary statistics... Ok!  Chain for summary
statistics: A
  File with the PDB sequence for summary statistics: default
> Computing summary statistics from the simulated data "./sequences*"..
  "./ProteinEvolverABC_Phase4.sh" was detected... Ok!
  "./ArgumentsSS.txt" was detected... Ok!
SSs dataset: "./sequences000000286"
ERROR: Too few informative sites to test significance.  Try decreasing
windowsize or increasing alignment length


Exiting...
rm: Phi.poly.unambig.sites: No such file or directory
SSs dataset: "./sequences000000500"

> Concatenating summary statistics from each simulated data ..

> Summary statistics placed in the file "./SSsimulations.ss"

rm: structures.in: No such file or directory
> Checking output files and preparing files for the ABC estimation phase..

Working on files "./ProteinEvolverABC_Pop_arguments.txt" and
"./SSsimulations.ss"
Simulations with incomplete calculation of Summary Statistics will be
skipped..
skipped line(s): 286
Done!

> Successful!
```

Simulations for which it was not possible to compute all the summary statistics are skipped (e.g. due to division by zero errors, etc., most of times caused by not enough genetic diversity in the simulated dataset). This would appear during the execution of this step,

```
SSs dataset: "./sequences000000036"
ERROR: Too few informative sites to test significance.  Try decreasing
windowsize or increasing alignment length


Exiting...
rm: Phi.poly.unambig.sites: No such file or directory
```

Skipped simulations will not be considered in the estimation phase. This may not be a problem for the estimation if the number of non-skipped simulations is large. In case of many skipped simulations, one should revise the prior distributions (unrealistic prior distributions could be problematic, for example if simulated data do not display sufficient substitution events) and/or increase the number of simulations to ensure a proper sampling of the parametric space.

A warning like the following is very common in some OS if a high number of simulations is performed,

```
SSs dataset: "./sequences000030140" sh: /bin/chmod: Argument list too long
```

This warning does not affect the calculations and can be ignored.

Depending on the specifications in the settings file, simulated data can be either removed or compressed into a file "SimulatedData.tar.gz" that will be stored in the "OtherOutputs" output folder.

-> Step 5 of *ProteinEvolverABC* is executed with *ProteinEvolverABC_Phase5.pl* and indicating the settings file. This script computes ABC estimations of substitution and recombination rates in the target multiple alignment of protein sequences by considering the previously obtained summary statistics of the simulated and input target datasets,

```
Example1_JTT_20SS_rapid % perl ProteinEvolverABC_Phase5.pl Settings.txt

*******************************************************************************
**********************************
ProteinEvolverABC_Phase5.pl
ProteinEvolverABC Phase 5 does:
- Read Settings file
- Perform ABC analysis using simulated data and target data (requires R with
library: abc)
*******************************************************************************
**********************************

> Reading Settings from input file ...
  Number of simulations: 500
  PDB file for summary statistics: 8C0G.pdb
    ./8C0G.pdb was detected for summary statistics... Ok!  Chain for summary
statistics: A
  File with the PDB sequence for summary statistics: default
  SummaryStatistics: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
  Number of iterations (simulations considered for the estimation phase): 450
  Tolerance number: 100
  ABCMethod: loclinear
  ABCTransf: logit
  ABCHCorr: 0
  MultiPage: 1

> Writing R output file "./PerformABC.r" ...

> R output file created: "./PerformABC.r" ...

> Performing ABC...
This phase requires the programming language R with the library: abc
 Summary Statistics file read
 Parameters file read
 Sequences data file read

> ABC method performed, printing results ...

Estimates from the rejection step:
                     Rho       Theta
Min.:          0.7671828  60.9038144
2.5% Perc.:    1.4562591  82.1966465
Median:       35.5191738 189.9129803
Mean:         37.9484443 190.8523943
Mode:         22.4918245 183.4978948
97.5% Perc.:  77.7674285 289.0611536
Max.:         79.6766116 295.2197694

Estimates from the local linear regression step:
                            Rho      Theta
Min.:                  2.202735   5.588878
Weighted 2.5 % Perc.:  2.497681 141.131947
```

```
Weighted Median:          6.317289 280.150425
Weighted Mean:            8.914872 262.873130
Weighted Mode:            5.460616 288.291620
Weighted 97.5 % Perc.:   28.961338 297.579547
Max.:                    72.422932 298.401210

 Plots of posterior distributions from the rejection step created
 Plots of posterior distributions from the local linear regression step
created
 Plots to assess ABC analysis integrity created

> Successful!
```

Regarding the Local Lineal Regression algorithm for the ABC estimation, too restrictive tolerance levels may arise difficulties. It is convenient exploring an optimal level of tolerance as presented in the associated article. We recommend use 1% or a lower level of tolerance (i.e., for 50000 simulations, explore *ABCTolerance=500* or *ABCTolerance=100*). *ABCTolerance* below 100 can be problematic due to involving little information for the estimations.

Interestingly, for the ABC estimation phase, not all the previously calculated summary statistics have to be applied to perform the estimation. A preliminary exploration of the more informative summary statistics for the estimates is recommended, as presented in the associated study. Importantly, the step 5 can be run multiple times (for each time, the parameters for the ABC estimation in the settings file can be previously modified, for example the subset of summary statistics used for the estimation). Further details are shown later.

3.4. Output files

Several output files are generated by *ProteinEvolverABC* during the different stages of the estimation. These files are saved into the output folders "*MainOutputs*" and "*OtherOutputs*" that will be placed in the working directory.

3.4.1. Output files from simulations

A variety of information is saved from the simulations in the working directory, output folder "*OtherOutputs*":

- The file "*ProteinEvolverABC_arguments.txt*" shows the *ProteinEvolver* arguments used to perform the simulations (further details in the *ProteinEvolver* documentation, https://github.com/MiguelArenas/proteinevolver). The file "*ProteinEvolverABC_Pop_arguments.txt*" includes the same information but considering the population recombination ρ and population amino acid substitution θ rates. The file "*PSimulations.csv*" shows the values sampled from the prior distributions for the parameters to be estimated (ρ and θ).
- The simulated alignments are compressed into the file "*SimulatedData.tar.gz*" if the user specified in the settings file the option to save the simulations.
- The files "*Histogram_PriorRecombination.pdf*" and "*Histogram_PriorSubstitution.pdf*" show illustrative histograms of the prior distributions used for the simulations of recombination and substitution rates, respectively. In addition, histograms are also provided for ρ and θ.
- The summary statistics computed from each simulated data are printed in the files "*SSsimulations.ss*" and "*SSsimulations.csv*".

- Some temporal scripts and outputs (namely, *MakePriorsProteinEvolverABC.r*, *ProteinEvolverABC_Phase2.sh*, *ProteinEvolverABC_Phase3.sh, Output_MakePriorsProteinEvolverABC.txt*, *PerformABC.r*) are also stored in the folder "*OtherOutputs*". These scripts can be helpful for debugging any running-error of *ProteinEvolverABC* (e.g., checking errors from incorrect specification of parameters for the simulations).
- Several temporal intermediate files necessary for the run of SCS models are stored in the folder "OtherOutputs". In the case of applying the substitution model SSCPE, for example, several outputs related to internal calculations and the site-specific exchangeability matrices are generated, and stored in the folder "OutputsSSCPE" that is moved to the folder "OtherOutputs". In the case of applying the models ProtASR2_MF or ProtASR2_WT, a folder named "Outputs_ProtASR2" is crated to store the same kind of intermediate files, for example related to stability and exchangeability matrices, and again moved to the folder "OtherOutputs".

3.4.2. Summary statistics from the target multiple alignment of protein sequences

- The summary statistics computed from the target multiple alignment of protein sequences are printed in the file "*SSRealData.ss*" and saved in the folder "*OtherOutputs*".
- Considering the possibility to re-run multiple times the ABC estimation with changes in the parameters of the settings file, a file named *SSRealDataSelected.ss* is created for each run, containing the used summary statistics.

3.4.3. Output files from the estimations

The information on the ABC estimations is saved in the folder "*MainOutputs*":

- The file "*Histograms_SStats.pdf*" shows a histogram of the values of summary statistics from the retained simulations for each summary statistic considered. Vertical blue lines corresponding to the value of the summary statistic of the target sequences alignment are added. Ideally the histograms should have a Gaussian-like shape with the blue line in the centre of the distribution. These plots are used to assess if the model assumed for the simulations fits well the data.
- The file "*Scaterplots_SStatsVSParams.pdf*" shows a scatter plot of the values of the summary statistics and corresponding values of the parameters assumed for the simulation (i.e. recombination and substitution rates) for each pair of considered summary statistic and parameter. Again, a blue line corresponding to the value of the summary statistic of the target sequences alignment is added. Ideally the blue line should be somewhere in the centre of the y-axis. These plots can give a rough visualization of the local linear univariate relation between the parameters and the summary statistics. Note, however, that the adjustment via the regression-step is performed via a multiple regression, which considers all the summary statistics together.
- The file "*PCA_SStats.pdf*" shows a scatter plot of the two first principal components of a principal component analysis (PCA) on the considered summary statistics. The black points correspond to a sample of all the

simulations, the red points correspond to all the retained simulations, and the yellow point correspond to the target sequences alignment. Ideally the red area should be well inside the black area and the yellow point should be well inside the red area. This plot is also used to assess if the model assumed for the simulations fits well the data.

- The files "*Posteriors_rejection.pdf*" and "*Posteriors_loclinear.pdf*" show the posterior distributions of the estimated parameters for the *regression* and *rejection* methods, respectively. Black lines represent the prior distributions, and they are obtained by considering a random sample of the simulations. Histograms and their smooth curve representation (blue lines) represent the posterior distributions.
- The files "*Summary_ABC_rejection.csv*" and "*Summary_ABC_loclinear.csv*" present the estimations of recombination and substitution rates under the *rejection* and *regression* methods, respectively. These files are in a "comma separated values" formatted and can be easily opened by excel or other spreadsheet software.

3.5. Re-analysing data

The *ProteinEvolverABC* package allows perform the estimation with different settings for the ABC estimation step (Step 5). In particular, these settings can be changed in the settings file for a better tuning of the estimations and repeat the execution of the step 5 (*perl ProteinEvolverABC_Phase5.pl Settings.txt*). Among these settings, the most important considerations are the following:

- ABC iterations. This setting defines the number of simulations to consider in the ABC analysis, and cannot be higher than the total number of simulations performed in previous steps. By considering different values of this setting, the user may get some insights on how well the entire parameters space is characterized in the simulations. For estimations using a target dataset that was simulated, in which one can assume for the simulations the exact model that originated such a target alignment, we showed that 50,000 simulations are enough for consistent estimations. However, when considering "real" target sequences alignments, many more points may be required (e.g. 100,000 simulations).

- ABC tolerance. This setting defines the number of simulations closest to the target sequences alignment that are retained for the estimation. Choosing the tolerance is not trivial: it has to be large enough so that to provide a good characterization of the posterior distribution, but on the other hand, it has to be small enough so that the simulations retained are not very far from the target sequences alignment. The use of the regression-step adds another concern: performing multiple regressions using a small number of points can introduce some problems. For estimations using simulated target alignments we found that a tolerance of 100 simulations (i.e., 1% of the total number of simulations) worked fine for several explored total number of simulations. However, when considering "real" target alignments, a wider tolerance could be needed (e.g. 1,000 simulations).

- ABC transformation. This setting defines which, if any, transformation will be performed to the parameter values before adjusting them with the regression method. These transformations have the advantage of ensuring that the regression-adjustment

does not project points outside the region of interest. Particularly, the "log" transformation ensures that the adjusted values of the retained simulations are positive, while the "logit" transformation ensures that they are inside the prior distribution range.

- ABC correction. This setting defines whether to correct for the heteroscedasticity of the parameters when performing the regression adjustment. This correction was introduced by Blum and Francois [31] and was shown to have considerable impact on the ABC estimations in particular cases [31].

- Summary statistics to use. This setting defines which summary statistics to consider when performing ABC estimations. As the number of simulations and the tolerance interval, the choice on the summary statistics is not trivial in ABC. For the estimation of recombination and substitution rates, we showed that the use of all the summary statistics provided in *ProteinEvolverABC* works fairly well.

Due to the advantages of re-analysing the target dataset with different settings of the ABC estimation, we suggest a simple way to perform these re-analysis:
1. Rename the "*MainOutputs*" folder.
2. Create a new settings file with different ABC settings.
3. Re-run step 5 of the *ProteinEvolverABC* pipeline with the new settings file.
Note, however, that you should keep track on which settings file correspond to which renamed "*MainOutputs*" folder.

3.6. Examples

The package includes the following examples in the folder "*examples*" including all input and output files:

- **Example1_JTT_16SS_rapid:** Analysis of a real target alignment [11 sequences, 289 amino acids using only 500 simulations (a few simulations with the aim of only performing a rapid exploration of the framework and so results may not be accurate) with the following prior distributions [ρ = Uniform(0,80) and θ: = Uniform(0,300)] while assuming an effective population size $N = 1000$. The simulations used the JTT empirical substitution model of protein evolution and 16 summary statistics only based on sequence information (information from the protein structure is not used). The simulations were not saved to save space in the hard disk.
- **Example2_JTT_20SS_rapid:** Analysis of a real target alignment [11 sequences, 289 amino acids using only 500 simulations (a few simulations with the aim of only performing a rapid exploration of the framework and so results may not be accurate) with the following prior distributions [ρ = Uniform(0,80) and θ: = Uniform(0,300)] while assuming an effective population size $N = 1000$. The simulations used the JTT empirical substitution model of protein evolution and 20 summary statistics based on both sequence and structural information. The simulations were not saved to save space in the hard disk.
- **Example3_SCS_WT_20SS_rapid:** Analysis of a real target alignment [11 sequences, 289 amino acids using only 500 simulations (a few simulations with the aim of only performing a rapid exploration of the framework and so results may not be accurate) with the following prior distributions [ρ = Uniform(0,80) and θ: = Uniform(0,300)] while assuming an effective population size $N = 1000$. The simulations used the SCS WT substitution model of protein evolution

implemented in *ProtASR2*, and 20 summary statistics based on both sequence and structural information. The simulations were not saved to save space in the hard disk.

- **Example4_SCS_SSCPE_20SS_rapid:** Analysis of a real target alignment [11 sequences, 289 amino acids using only 500 simulations (a few simulations with the aim of only performing a rapid exploration of the framework and so results may not be accurate) with the following prior distributions [ρ = Uniform(0,80) and θ: = Uniform(0,300)] while assuming an effective population size $N = 1000$. The simulations used the SCS SSCPE substitution model of protein evolution implemented in the tool *SSCPE*, and 20 summary statistics based on both sequence and structural information. The simulations were not saved to save space in the hard disk.
- **Example5_LigD_N:** Analysis of a real target alignment [133 sequences, 108 amino acids using only 500 simulations (a few simulations with the aim of only performing a rapid exploration of the framework and so results may not be accurate) with the following prior distributions [ρ = Uniform(0,43) and θ: = Uniform(0,242)] while assuming an effective population size $N = 1000$. The simulations used the SCS MF substitution model of protein evolution implemented in *ProtASR2*, and 20 summary statistics based on both sequence and structural information. The simulations were not saved to save space in the hard disk.
- **Example6_Metiltransferase:** Analysis of a real target alignment [11 sequences, 289 amino acids using only 5000 simulations (a few simulations with the aim of only performing a rapid exploration of the framework and so results may not be accurate) with the following prior distributions [ρ = Uniform(0,80) and θ: = Uniform(0,300)] while assuming an effective population size $N = 1000$. The simulations used the SCS WT substitution model of protein evolution implemented in *ProtASR2*, and 20 summary statistics based on both sequence and structural information. The simulations were not saved to save space in the hard disk.

Note that these examples are based on a small number of simulations (i.e., 500 or 50000), a more accurate analysis would imply the use of 50,000 simulations.

3.7. Message errors and recommendations

Errors generated from incorrect settings are usually shown on the screen and the program will suggest abort the execution by typing CTRL+C. **For beginners, I recommend copying a provided input file "Settings.txt" from the "examples" folder and edit it with the desired settings.**

Running Phase 1

The input multiple sequence alignment must be presented in *standard phylip sequential* format (section 4.2). In order to avoid errors with the format, we recommend use the format alignment converter *ALTER* (http://sing.ei.uvigo.es/ALTER/) [29]. Therein after uploading your input alignment, go to "*Select program*" and in the section "*Population genetics*" please select "*CodABC*". Then select "*PHYLIP*" and finally, covert.

Importantly, the amino acid sequences of the input multiple sequence alignment should only include any of the 20 amino acids (one-letter code) or indels (as "-"). Other letters or symbols (i.e., X, $, ., etc) will produce an error displayed on the screen.

Running Phase 2

If simulations start but stop before the R execution, check that the R libraries (lattice, MCMCpack, ape, graphics, abc) are correctly installed (see section 3.1). The R function "suppressPackageStartupMessages" should also be installed for this purpose. Also check that you have compiled *ProteinEvolverABC* and the executable files appear in the "*bin*" folder.

We noted that running the SSCPE model can have problems running on certain OSs (i.e., MacOS) but it works in Linux. In case this aspect cannot be solved, we recommend using the SCS models WT (*ProtASR2_WT*) and MF (*ProtASR2_MF*).

If SCS models start but stop before generating the exchangeability matrix, several issues should be considered. First, some constraints are imposed by the model over extremely unrealistic (based on ΔG calculations) or poorly aligned representative structures. If the sequence of the PDB does not align well with any of the sequences of the target multiple sequence alignment, the run will be aborted. Also, it might fail because of a temporary lack of memory, so make sure that the available resources are enough. It will depend on the number and length of the protein sequences and the size of the protein structure. Moreover, check that the compilation of the program was done in the same working directory that it will be executed. If executable files are moved after compiling in between phases, the SCS model run might fail. In case it happened, compiling again could solve the problem.
The calculation of summary statistics for the target multiple sequence alignment could fail for the structural summary statistics (17-20) due to lack of information (i.e., the protein structure or its sequence is not given).

Running Phase 3

If simulations start but stop after the R execution, check the documentation of *ProteinEvolver* but most likely a setting is incorrectly specified (i.e., a negative substitution rate).

In general, note that the simulation phase can be computational costly with long running times. This is particularly relevant when performing many simulations, when the simulated data includes a large number and long sequences, or when considering high recombination rates. Reducing the number of simulations and the size of the input alignment of protein sequences can dramatically reduce computer time. Sometimes reducing the input dataset is not possible and then one can try to reduce the number of simulations. To do so, one can reduce the prior distribution of the recombination and substitution rates (assuming the user has an idea about the possible estimates of these parameters) to have an enough number of simulations covering the desired parametric landscape. Indeed, running the simulations in parallel (if the machine has more than one processor) can reduce computer times.

Another concern is the lack of RAM memory when running the computer simulations. This can occur in scenarios with a user-specified very high recombination rate, when using SCS models and for data with many and long amino acid sequences. Regarding recombination, it increases the number of nodes of the ARG (ancestral recombination graph [25, 26]). Thus, when ρ is very high (e.g. above 150), the ARG needs to save in memory a high amount of information. As the simulation time exponentially increases with ρ, it may lead to lack of available memory, and the program stops with an error

message. In this situation we recommend reduce the user-specified values of ρ (in its prior distribution) or population size. Of course, another option is just using a machine with more RAM memory.

Running Phase 4

When computing the summary statistics from the simulations one can find that for some simulations one or more summary statistics cannot be computed (e.g. due to division by zero errors, etc). For example, the Phi summary statistics cannot be computed if there is not an enough number of substitutions in the input data and one can find on the screen a text like,

```
SSs dataset: "./sequences000000036"
ERROR: Too few informative sites to test significance.  Try decreasing
windowsize or increasing alignment length


Exiting...
rm: Phi.poly.unambig.sites: No such file or directory
```

This is not problematic, those simulations will be automatically excluded for the estimation phase.

Indeed, the calculation of summary statistics based on the protein structure requires more computational time. Also, the computation of summary statistics can take a long time if there are many simulated data and in the simulated data is large (many and long sequences). Fortunately, this version of the program includes the possibility to parallelize the step 4, partially relieving the time-cost inconveniences.

Running Phase 5

One can find that the estimation with the regression approach could not be performed. In this situation this phase can be executed again (as previously indicated) varying the ABC tolerance in the input settings file. See details later about the rejection and regression approaches. The rejection approach can be more robust (in terms of being able to work) than the regression approach to analyze real data but in theory the regression approach can (not always) provide more accurate estimates than the rejection approach due to its complexity. In addition to varying the tolerance, one can increase the number of simulations to obtain results from the regression approach, but this implies repeat all the phases (1-5) with the new number of simulations.

One can also find that the results from the rejection approach can disagree with the results from the regression approach. This can occur especially when analyzing real data and there is not a direct answer to decide which approach is more accurate for the study dataset [32]. For a given real dataset the rejection approach can be more accurate than the regression approach, and *vice versa* for another real dataset. As indicated above, the rejection approach is more robust (in terms of being able to work) than the regression approach to analyze real data but in theory the regression approach can (not always) provide more accurate estimates than the rejection approach. Therefore, the basis could be using the results from the rejection approach if the results from the regression approach are not available. A formal procedure to select the estimation method would be performing computer simulations (mimicking the real dataset) where the true values of the parameters under study (sampled from the corresponding prior distributions) are known, but this can take a long time and also the simulated data could still differ from the real data (i.e., if the real data suffered a complex evolutionary process and presents a complex pattern of molecular diversity, which is a limitation of ABC). In the associated

study we evaluated for ABC estimation approaches and this can provide recommendations for users.

If you find any unexpected error, or there is any doubt, do not hesitate to contact marenas@uvigo.es. Thanks for your contribution!

4. Models and methods

ProteinEvolverABC is a framework to estimate recombination and substitution rates from protein sequence alignments using an ABC procedure. The simulator includes a variety of evolutionary models that may help to better mimic real scenarios. Then, informative summary statistics are calculated from the simulated data. Finally, the computation of the posterior distributions is performed using ABC methods.

4.1. Evolutionary models and computer simulations

The first step of *ProteinEvolverABC* is based on the simulator *ProteinEvolverProtABC*, which is an adapted version of the program *ProteinEvolver* (https://github.com/MiguelArenas/proteinevolver) [3] to perform ABC.
ProteinEvolverProtABC implements a variety of evolutionary models and generates protein sequence alignments (see below) collected at same or different times (temporal longitudinal sampling or tip dates [see, 33]), from a population evolved under complex recombination by using the coalescent approach [34-36]. Protein sequences are evolved under a variety of empirical substitution models of protein evolution *Blosum62* [37], *CpRev* [38], *Dayhoff* [39], *DayhoffDCMUT* [40], *HIVb* [41], *HIVw* [41], *JTT* [42], *JonesDCMUT* [40], *LG* [43], *Mtart* [44], *Mtmam* [45], *Mtrev24* [46], *RtRev* [47], *VT* [48], *WAG* [49] and any user-defined model, either global (*UserEAAM*) or site-specific (*UserEAAMSites*).
In addition, the framework includes several SCS models of protein evolution, including the SSCPE model [4] and the SCS models implemented in the program *ProtASR2* [5], namely the wild-type (WT) model (*ProtASR2_WT*) and the meanfield model (*ProtASR2_MF*).
Both haploid and diploid data can be simulated. Indeed, the program also implements heterogeneity across sites according to a gamma distribution (+G) [50] and proportion of invariable sites (+I) [51].

4.2. Summary statistics

The package implements a total of 20 summary statistics (Table 2).
Three of the summary statistics are fast statistical tests for detecting recombination: the maximum chi-squared method [χ^2, [52]], the neighbor similarity scope [*NSS*, [53]], and the pairwise homoplasy index [*PHI*, [21]], available in the *PhiTest* software [21]. Other calculated summary statistics are the mean, standard deviation (sd), skewness (sk) and kurtosis (ku) of the amino acid (aa) diversity, aa heterozygosity (H) and aa pairwise sequence identity (si). Another summary statistic is the number of aa segregating sites (S). The next four summary statistics are related to statistical measurements of folding stability (ΔG), as follows: average ΔG, standard deviation of ΔG, skewness and kurtosis of ΔG.

Table 2. The implemented summary statistics, their code and their identifiers (Id).

Description	Code	Id
pairwise homoplasy index PHI	Phi	1
neighbor similarity scope NSS	NSS	2
maximum chi-squared χ^2	ChiSq	3
mean of aa diversity	p_av	4
sd of aa diversity	p_sd	5
sk of aa diversity	p_sk	6
ku of aa diversity	p_ku	7
mean of aa heterozygosity	H_av	8
sd of aa heterozygosity	H_sd	9
sk of aa heterozygosity	H_sk	10
ku of aa heterozygosity	H_ku	11
number of aa segregating sites	S	12
mean of pairwise aa sequence identity	si_av	13
sd of pairwise aa sequence identity	si_sd	14
sk of pairwise aa sequence identity	si_sk	15
ku of pairwise aa sequence identity	si_ku	16
mean of protein folding stability (ΔG)	av_dg	17
sd of protein folding stability (ΔG)	sd_dg	18
sk of protein folding stability (ΔG)	sk_dg	19
ku of protein folding stability (ΔG)	ku_dg	20

4.3. ABC estimation approaches

Two ABC methods are implemented in *ProteinEvolverABC*, a simple rejection method [12] and a weighted multiple linear regression method as presented in [1]. The detailed algorithms of these methods are described below (further details can be found in the references given, as well as in various review papers such as [32] or [54]).
These methods are performed using the R package "*abc*", further details on their implementation can be seen in [55].

4.3.1. The rejection approach

The basic algorithm, as introduced by Pritchard and co-workers [12] can be written as following:

(1) Sample (vector valued) parameter, Φ, from the prior: $\Phi_i \sim p(\Phi)$;
(2) Simulate data, D, given Φ_i : $D_i \sim p(D|\Phi_i)$;
(3) Summarize D_i with a set of chosen summary statistics to obtain S_i; go to (1) until N sample points from the joint distribution $p(S, \Phi)$ have been created;
(4) Accept δ points whose S_i have the smallest distance to s', the real data summarized by the same set of summary statistics.

In steps (1) to (3), one simulates independent pairs (Φ_i, S_i), i = 1, 2, …, N, where each Φ_i is an independent draw from the prior distribution of Φ, and S_i are values that summarize simulated values of D with $\Phi = \Phi i$. The (Φ_i, S_i) are then random draws from the joint density. Step (4), the rejection-step, provides an estimate of the conditional density, when $S = s'$, that is, the posterior distribution $p(\Phi|S = s')$. The idea is that the Φ_i for which $|S_i - s'|$ is small forms an approximation of a random sample from the desired posterior distribution $p(\Phi|D = d')$.

4.3.2. The regression approach

In the regression method two changes are proposed at step (4): smooth weighting and regression adjustment. The aim is to improve the sampling of the posterior density by weighting the Φ_i according to its distance from the real data by evaluating $|S_i - s'|$ and to adjust the Φ_i using a local-linear regression to weaken the effect of that discrepancy [1].

In the last step (4), it applies a weighting scheme to the points according to their distance to the s'. And then perform a linear multiple regression with points assigned non-zero weight. Adjust them according to $\Phi_i^* = \Phi_i + R_i$, where R_i is the residual of the ith point.

As noted by Beaumont and co-workers [1] the rejection method can be viewed as a special case of the local-linear regression approach when using a uniform kernel and a local-constant regression. Actually, as δ tends to zero, the regression and rejection methods become equivalent.

4.4. ProteinEvolverABC performance

There is a lack of evolutionary frameworks for the estimation of recombination rate in protein sequences, which does not allow a direct comparison of *ProteinEvolverABC* with other frameworks. Still, the methodology used by *ProteinEvolverABC* but applied to DNA coding sequences showed more accurate estimates of the recombination rate than common likelihood-based tools such as *OmegaMap* [16] or *Lamarc* [15], see [6].

In the associated article, we performed evaluations of *ProteinEvolverABC* using computer simulations with *ProteinEvolver* [3] under different substitution models (including empirical and SCS models) and summary statistics (including summary statistics that consider and that ignore the protein structure). The results indicated that both parameters can be accurately estimated using 50,000 computer simulations. As expected, the recombination rate can be more difficult to estimate at low levels of substitution rate, although this effect can be reduced by increasing the number of simulations (i.e., using 100,000 simulations). For further details see the article associated to this framework

A main advantage of *ProteinEvolverABC* is being robust to influences of recombination on substitution estimates, and *vice versa*. This marks a difference between analytical tools based on ABC and those based on other approaches [1, 6, 32, 56].

4.5. Assumptions and limitations

ABC assumes that simulated data can mimic the real data. Simulations are based on models and are never fully representative of the real process. This is a limitation present in any estimation approach based on a model of evolution. In this concern, the aim is to apply as realistic as possible simulations. The data-generating process assumed by *ProteinEvolverABC* consists of drawing a sample genealogy from the standard coalescent with recombination [35, 57], and then, given the genealogy, generating the sequences evolved under a substitution model of protein evolution [3]. This is a well-established methodology to simulate protein sequences upon evolutionary histories [58].

Recombination rate is a parameter that can be problematic to estimate in samples with low substitution rates. Indeed recombination events can be difficult to detect if there is not enough diversity in the data [59]. Thus, simulated data with low substitution rates may not reflect their recombination rates, which can complicate estimations of the latter parameter [60]. Nevertheless, in contrast to other analytical approaches, ABC can account for this uncertainty and reduce the bias in recombination estimates imposed by low substitution rates [6].

The second version of *ProteinEvolverABC* considers information about the protein structure through several SCS models and in some summary statistics. This was considered as recombination events, in addition to substitution events [30, 61], can affect the protein structure [10] and provide evolutionary signatures that can be useful to detect recombination and substitution rates.

5. Acknowledgments

- The first version of this framework was supported by the Spanish Ministry of Economy and Competitiveness and Ministry of Science and Innovation through the Grants [RYC- 2015-18241] and [PID2019-107931GA-I00/AEI/10.13039/501100011033].
- The second (current) version of this framework was partially supported by the grants [PID2023-151032 NB-C21] and [PID2023-151032NB-C22] funded by the Spanish Ministry of Science MCIU/the Spanish Agency of Research AEI/10.13039/501100011033/ and FEDER, UE.

6. References

1. Beaumont MA, Zhang W, Balding DJ: **Approximate Bayesian computation in population genetics**. *Genetics* 2002, **162**(4):2025-2035.
2. Beaumont MA: **Approximate Bayesian Computation**. *Annual Review of Statistics and Its Application* 2019, **6**(1):379-403.
3. Arenas M, Dos Santos HG, Posada D, Bastolla U: **Protein evolution along phylogenetic histories under structurally constrained substitution models**. *Bioinformatics* 2013, **29**(23):3020-3028.

4. Lorca-Alonso I, Otero-de-Navascues F, Arenas M, Bastolla U: **Site-Specific Structure and Stability Constrained Substitution Models Improve Phylogenetic Inference**. *Syst Biol* 2026, **75**(3):581-604.
5. Arenas M, Bastolla U: **ProtASR2: Ancestral reconstruction of protein sequences accounting for folding stability**. *Methods Ecol Evol* 2020, **11**(2):248-257.
6. Lopes JS, Arenas M, Posada D, Beaumont MA: **Coestimation of Recombination, Substitution and Molecular Adaptation rates by approximate Bayesian computation**. *Heredity* 2014, **112**(3):255-264.
7. Arenas M, Lopes JS, Beaumont MA, Posada D: **CodABC: A Computational Framework to Coestimate Recombination, Substitution, and Molecular Adaptation Rates by Approximate Bayesian Computation**. *Mol Biol Evol* 2015, **32**(4):1109-1112.
8. Arenas M, Sanchez-Cobos A, Bastolla U: **Maximum likelihood phylogenetic inference with selection on protein folding stability**. *Mol Biol Evol* 2015, **32**(8):2195-2207.
9. Minning J, Porto M, Bastolla U: **Detecting selection for negative design in proteins through an improved model of the misfolded state**. *Proteins* 2013, **81**(7):1102-1112.
10. Del Amparo R, González-Vázquez LD, Rodríguez-Moure L, Bastolla U, Arenas M: **Consequences of Genetic Recombination on Protein Folding Stability**. *Journal of Molecular Evolution* 2023, **91**(1):33-45.
11. Bastolla U, Arenas M: **The Influence of Protein Stability on Sequence Evolution: Applications to Phylogenetic Inference**. In: *Computational Methods in Protein Evolution.* Edited by Sikosek T. New York, NY: Springer New York; 2019: 215-231.
12. Pritchard JK, Seielstad MT, Perez-Lezaun A, Feldman MW: **Population growth of human Y chromosomes: a study of Y chromosome microsatellites**. *Mol Biol Evol* 1999, **16**(12):1791-1798.
13. Arenas M, Araujo NM, Branco C, Castelhano N, Castro-Nallar E, Perez-Losada M: **Mutation and recombination in pathogen evolution: Relevance, methods and controversies**. *Infect Genet Evol* 2018, **63**:295-306.
14. Arenas M, Lorenzo-Redondo R, Lopez-Galindez C: **Influence of mutation and recombination on HIV-1 in vitro fitness recovery**. *Mol Phylogenet Evol* 2016, **94**(Pt A):264-270.
15. Kuhner MK: **LAMARC 2.0: maximum likelihood and Bayesian estimation of population parameters**. *Bioinformatics* 2006, **22**(6):768-770.
16. Wilson DJ, McVean G: **Estimating diversifying selection and functional constraint in the presence of recombination**. *Genetics* 2006, **172**(3):1411-1425.
17. McVean GA, Myers SR, Hunt S, Deloukas P, Bentley DR, Donnelly P: **The fine-scale structure of recombination rate variation in the human genome**. *Science* 2004, **304**(5670):581-584.
18. Martin DP, Murrell B, Golden M, Khoosal A, Muhire B: **RDP4: Detection and analysis of recombination patterns in virus genomes**. *Virus Evol* 2015, **1**(1):vev003.
19. Arenas M, Posada D: **Coalescent simulation of intracodon recombination**. *Genetics* 2010, **184**(2):429-437.
20. Arenas M, Posada D: **The effect of recombination on the reconstruction of ancestral sequences**. *Genetics* 2010, **184**(4):1133-1139.

21. Bruen TC, Philippe H, Bryant D: **A simple and robust statistical test for detecting the presence of recombination**. *Genetics* 2006, **172**(4):2665-2681.
22. Sali A, Blundell TL: **Comparative protein modelling by satisfaction of spatial restraints**. *J Mol Biol* 1993, **234**(3):779-815.
23. Waterhouse A, Bertoni M, Bienert S, Studer G, Tauriello G, Gumienny R, Heer FT, de Beer TA P, Rempfer C, Bordoli L *et al*: **SWISS-MODEL: homology modelling of protein structures and complexes**. *Nucleic Acids Research* 2018, **46**(W1):W296-W303.
24. Jumper J, Evans R, Pritzel A, Green T, Figurnov M, Ronneberger O, Tunyasuvunakool K, Bates R, Žídek A, Potapenko A *et al*: **Highly accurate protein structure prediction with AlphaFold**. *Nature* 2021, **596**(7873):583-589.
25. Arenas M: **The Importance and Application of the Ancestral Recombination Graph**. *Front Genet* 2013, **4**:206.
26. Griffiths RC, Marjoram P: **An ancestral recombination graph**. In: *Progress in population genetics and human evolution.* Edited by Donelly P, Tavaré S, vol. 87. Berlin: Springer-Verlag; 1997: 257-270.
27. Iglesias-Rivas P, Del Amparo R, Cabaleiro JA, Arenas M: **Empirical substitution models of protein evolution: database, relationships, and modeling considerations**. *Database* 2025, **2025**:baaf052.
28. Yang Z: **PAML 4: phylogenetic analysis by maximum likelihood**. *Mol Biol Evol* 2007, **24**(8):1586-1591.
29. Glez-Pena D, Gomez-Blanco D, Reboiro-Jato M, Fdez-Riverola F, Posada D: **ALTER: program-oriented conversion of DNA and protein alignments**. *Nucleic Acids Res* 2010, **38**(Web Server issue):W14-18.
30. Ferreiro D, Pazos E, Arenas M: **Trends in substitution models of protein evolution for phylogenetic inference**. *Mol Phylogenet Evol* 2026, **214**:108473.
31. Blum MGB, François O: **Non-linear regression models for Approximate Bayesian Computation**. *Statistics and Computing* 2010, **20**:63-73.
32. Beaumont MA: **Approximate Bayesian Computation in Evolution and Ecology**. *Annu Rev Ecol Evol Syst* 2010, **41**:379-405.
33. Navascues M, Depaulis F, Emerson BC: **Combining contemporary and ancient DNA in population genetic and phylogeographical studies**. *Mol Ecol Resour* 2010, **10**(5):760-772.
34. Kingman JFC: **The coalescent**. *Stochastic Processes and their Applications* 1982, **13**:235-248.
35. Hudson RR: **Properties of a neutral allele model with intragenic recombination**. *Theoretical Population Biology* 1983, **23**(2):183-201.
36. Hudson RR: **Generating samples under a Wright-Fisher neutral model of genetic variation**. *Bioinformatics* 2002, **18**(2):337-338.
37. Henikoff S, Henikoff JG: **Amino acid substitution matrices from protein blocks**. *Proc Natl Acad Sci U S A* 1992, **89**(22):10915-10919.
38. Adachi J, Waddell PJ, Martin W, Hasegawa M: **Plastid genome phylogeny and a model of amino acid substitution for proteins encoded by chloroplast DNA**. *J Mol Evol* 2000, **50**(4):348-358.
39. Dayhoff MO, Schwartz RM, Orcutt BC: **A model of evolutionary change in proteins**. In: *Atlas of protein sequence and structure.* Edited by Dayhoff MO, vol. 5, Suppl. 3. Washington D. C.; 1978: 345-352.
40. Kosiol C, Goldman N: **Different versions of the Dayhoff rate matrix**. *Mol Biol Evol* 2005, **22**(2):193-199.

41. Nickle DC, Heath L, Jensen MA, Gilbert PB, Mullins JI, Kosakovsky Pond SL: **HIV-specific probabilistic models of protein evolution**. *PLoS One* 2007, **2**(6):e503.
42. Jones DT, Taylor WR, Thornton JM: **The rapid generation of mutation data matrices from protein sequences**. *Comput Appl Biosci* 1992, **8**(3):275-282.
43. Le SQ, Gascuel O: **An improved general amino acid replacement matrix**. *Mol Biol Evol* 2008, **25**(7):1307-1320.
44. Abascal F, Posada D, Zardoya R: **MtArt: A New Model of Amino Acid Replacement for Arthropoda**. *Mol Biol Evol* 2007, **24**(1):1-5.
45. Yang Z, Nielsen R, Masami H: **Models of amino acid substitution and applications to mitochondrial protein evolution**. *Mol Biol Evol* 1998, **15**(12):1600-1611.
46. Adachi J, Hasegawa M: **MOLPHY version 2.3: programs for molecular phylogenetics based in maximum likelihood**. *Comput Sci Monogr* 1996, **28**:1-150.
47. Dimmic MW, Rest JS, Mindell DP, Goldstein RA: **rtREV: an amino acid substitution matrix for inference of retrovirus and reverse transcriptase phylogeny**. *J Mol Evol* 2002, **55**(1):65-73.
48. Muller T, Vingron M: **Modeling amino acid replacement**. *Journal of Computational Biology* 2000, **7**(6):761-776.
49. Whelan S, Goldman N: **A general empirical model of protein evolution derived from multiple protein families using a maximum-likelihood approach**. *Mol Biol Evol* 2001, **18**(5):691-699.
50. Yang Z: **Among-site rate variation and its impact on phylogenetic analysis**. *Trends Ecol Evol* 1996, **11**(9):367-372.
51. Fitch WM, Margoliash E: **A method for estimating the number of invariant amino acid coding positions in a gene using cytochrome c as a model case**. *Biochem Genet* 1967, **1**(1):65-71.
52. Smith JM: **Analyzing the mosaic structure of genes**. *J Mol Evol* 1992, **34**(2):126-129.
53. Jakobsen IB, Easteal S: **A program for calculating and displaying compatibility matrices as an aid to determining reticulate evolution in molecular sequences**. *Comput Appl Biosciences* 1996, **12**:291-295.
54. Blum MGB, Nunes MA, Prangle D, Sisson SA: **A comparative review of dimension reduction methods in approximate Bayesian computation**. *Stat Sci* 2013, **28**(2):189-208.
55. Csillery K, Francois O, Blum MGB: **abc: an R package for approximate Bayesian computation (ABC)**. *Methods Ecol Evol* 2012, **3**(3):475-479.
56. Bertorelle G, Benazzo A, Mona S: **ABC as a flexible framework to estimate demography over space and time: some cons, many pros**. *Mol Ecol* 2010, **19**(13):2609-2625.
57. Arenas M: **Computer programs and methodologies for the simulation of DNA sequence data with recombination**. *Front Genet* 2013, **4**:9.
58. Yang Z: **Molecular Evolution: A Statistical Approach**. Oxford, England.: Oxford University Press; 2014.
59. Posada D, Crandall KA: **Evaluation of methods for detecting recombination from DNA sequences: computer simulations**. *Proc Natl Acad Sci U S A* 2001, **98**(24):13757-13762.
60. Myers SR, Griffiths RC: **Bounds on the minimum number of recombination events in a sample history**. *Genetics* 2003, **163**(1):375-394.

61. Echave J, Spielman SJ, Wilke CO: **Causes of evolutionary rate variation among protein sites**. *Nat Rev Genet* 2016, **17**(2):109-121.